

N° d'ordre:

RÉPUBLIQUE ALGERIENNE DÉMOCRATIQUE ET POPULAIRE
MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR ET DE LA RECHERCHE SCIENTIFIQUE
UNIVERSITÉ DJILLALI LIABÈS DE SIDI BEL ABBÈS
FACULTÉ DES SCIENCES EXACTES
DÉPARTEMENT D'INFORMATIQUE



MÉMOIRE DE MASTER

Domaine : Mathématiques-informatique

Filière : Informatique

Spécialité : Réseaux, systèmes et sécurité de l'information (RSSI)

Par

M^R YOUSFI DJAMEL EDDINE

M^R BENCHIHA AKREM

SYSTÈME DE STATIONNEMENT INTELLIGENT EN UTILISANT LE FOG COMPUTING

Soutenu le ...-2023 devant le jury :

Dr. ADJOUDJ REDA	UDL-SBA	Président du jury
Dr. BOUKHARI NOUREDDINE	UDL-SBA	Examineur
Dr. BOUAMAMA SAMAH	UDL-SBA	Directeur de Mémoire

Année Universitaire : 2022 - 2023

À ma famille, dont l'amour indéfectible et le soutien ont été les piliers de mon parcours. Votre encouragement, vos sacrifices et votre foi en moi ont été ma motivation constante. Ce mémoire vous est dédié. À mon conseiller, dont les conseils et l'expertise ont façonné mes recherches et ma croissance académique. Votre mentorat, votre patience et votre dévouement ont été inestimables pour façonner le résultat de ce mémoire. Je suis reconnaissant des opportunités que vous m'avez offertes et de m'avoir poussé à atteindre mon plein potentiel. À mes amis et collègues, qui ont été une source d'inspiration et de camaraderie tout au long de cette entreprise académique. Votre encouragement, vos discussions et vos expériences partagées ont enrichi ma compréhension et ont rendu ce voyage plus significatif. Ce mémoire est dédié à tous ceux qui ont joué un rôle, qu'il soit grand ou petit, dans mon parcours académique. Vos contributions ont été importantes et ont façonné ce travail de manière profonde. Merci d'avoir été une partie de cette étape importante.

Yousfi Djamel Eddine

*Chère famille et Dr Bouachria Ahmed (paix à son âme), Je tiens à exprimer ma
profonde gratitude envers ma famille, qui continue de jouer un rôle essentiel
dans ma vie, ainsi que le rôle précieux du défunt Dr Bouachria Ahmed dans le
choix de cette spécialisation. Votre soutien constant et les conseils du Dr
Bouachria Ahmed ont été d'une importance capitale pour mon parcours
académique. Je suis reconnaissant d'avoir bénéficié de votre présence et de votre
influence. Merci infiniment pour tout ce que vous avez fait. Avec tout mon
amour et ma reconnaissance,*

Benchiha Akrem

REMERCIEMENTS

Nous sommes enchantés de consacrer ces quelques lignes pour exprimer notre gratitude et notre profonde reconnaissance envers tous ceux qui, de près ou de loin, ont contribué à la réalisation et à l'achèvement de ce travail. Tout d'abord, nous tenons à exprimer notre gratitude envers ALLAH pour nous avoir accordé le courage et la patience nécessaires pour mener à bien cette tâche. Nous souhaitons particulièrement remercier notre encadrante, Madame BOUAMAMA Samah, pour son soutien, sa rigueur, sa disponibilité, ses précieux conseils et son assistance tout au long de l'élaboration de ce travail. Nous remercions également Monsieur ADJOUDJ Reda (le président) d'avoir accepté de présider cette soutenance, ainsi que Monsieur BOUKHARI Noureddine d'avoir accepté d'examiner et d'évaluer ce mémoire. Enfin, nous souhaitons exprimer avec plaisir notre gratitude et nos remerciements à tous nos enseignants pour la qualité de l'enseignement qu'ils ont généreusement dispensé tout au long de nos études, nous offrant ainsi une formation efficace.

TABLE DES MATIÈRES

TABLE DES MATIÈRES	v
LISTE DES FIGURES	vii
LISTE DES TABLEAUX	viii
1 INTRODUCTION	1
2 GÉNÉRALITÉS SUR LE CLOUD , EDGE ET IoT	2
2.1 INTRODUCTION	3
2.2 LE CLOUD	3
2.2.1 Définition	3
2.2.2 Caractéristiques	4
2.2.3 Les modèles de services du Cloud	5
2.2.4 Les modèles de déploiement du Cloud	7
2.2.5 Les avantages et les inconvénients du Cloud	8
2.3 EDGE COMPUTING	10
2.3.1 Définition	10
2.3.2 Edge Computing : cas d'usage	10
2.3.3 Caractéristiques	10
2.3.4 Les avantages de l'Edge Computing	11
2.3.5 Internet des objets	12
2.3.6 Définition	12
2.3.7 Domaines d'applications de l'IoT	12
2.4 CONCLUSION	14
3 FOG COMPUTING	16
3.1 INTRODUCTION	17
3.2 DÉFINITION	17
3.3 CARACTÉRISTIQUES	17
3.4 LES AVANTAGES	19
3.5 LES INCONVÉNIENTS	19
3.6 COMPARAISON ENTRE LE CLOUD COMPUTING ET LE FOG COMPUTING	22
3.7 COMPARAISON ENTRE LE FOG COMPUTING ET LE EDGE COMPUTING .	23
3.8 CONCLUSION	25
4 ETAT DE L'ART	26
4.1 INTRODUCTION	27

4.2	ARCHITECTURES DU SYSTÈME DE FOG	27
4.2.1	Architectures agnostiques pour les applications d'utilisateur final des systèmes Fog	27
4.2.2	Architectures spécifiques aux applications pour les systèmes de Fog	30
4.3	ALGORITHMES DE POSITIONNEMENT DES DONNÉES SÉLECTIONNÉES	33
4.3.1	Algorithme Edge-Ward	33
4.3.2	Algorithme Cloud-Only	33
4.3.3	Algorithme de Mapping	33
4.4	SIMULATEUR DE FOG COMPUTING	34
4.4.1	Simulateur IFogSim	34
4.4.2	Simulateur FogTorch	34
4.4.3	Simulateur FogNetSim++	34
4.4.4	Comparaison entre les différents simulateurs de Fog	35
4.5	CONCLUSION	36
5	MODÈLE D'ARCHITECTURE PROPOSÉ ET SIMULATIONS	37
5.1	INTRODUCTION	38
5.2	PROBLEMATIQUES	38
5.3	MODELE D'ARCHITECTURE PROPOSEE	38
5.3.1	Architecture Cloud-Fog	39
5.3.2	Caméras	39
5.3.3	Les nœuds de Fog	39
5.3.4	La couche Cloud	40
5.4	SIMULATION	40
5.4.1	Outils utilisés	40
5.4.2	Présentation du simulateur IFogSim	44
5.4.3	Classes utilisées	44
5.4.4	Stratégies de placement	46
5.4.5	Topologie	46
5.4.6	Implementation avant les résultats	48
5.4.7	Evaluation des résultats	51
5.4.8	Conclusion	57
6	CONCLUSION GÉNÉRALE	58
	BIBLIOGRAPHIE	59

LISTE DES FIGURES

2.1	Les cinq caractéristiques du Cloud Computing.	5
2.2	SPI—Modèle d’offre de services du cloud.	5
2.3	Les Modèles de déploiement du cloud	8
2.4	L’impact économique potentiel des applications de l’IoT	13
5.1	Architecture à trois niveaux pour le système de stationnement intelligent	39
5.2	Interaction entre les caméras, les Fog nœuds et le Cloud Data Center	40
5.3	Interface de IFogSim	43
5.4	Topologie iFogSim de 9 caméras connectées à 3 nœuds de Fog connectés au serveur Cloud via le serveur proxy.	47
5.5	Topologie iFogSim d’un scénario basé sur isOnlyCloud avec des caméras connectées par l’intermédiaire d’un routeur.	48
5.6	Code principal	49
5.7	Création des appareils de brouillard (Fog Devices).	49
5.8	Création des routeurs.	50
5.9	Création des caméras.	50
5.10	Création des applications	51
5.11	Impact de la variation du nombre de cameras sur la latence dans les deux modèle proposés (isOnlyCloud) et (isOnlyFog).	52
5.12	Impact de la variation du nombre de cameras sur l’utilisation du réseau dans les deux modèle proposés (isOnlyCloud) et (isOnlyFog).	54
5.13	Impact de la variation du nombre de cameras sur le temps d’exécution dans les deux modèle proposés (isOnlyCloud) et (isOnlyFog).	55
5.14	Impact de la variation du nombre de cameras sur la consommation d’énergie dans les deux modèle proposés (isOnlyCloud) et (isOnlyFog).	56

LISTE DES TABLEAUX

2.1	La différence entre les services Cloud	7
3.1	Tableau comparative entre les caratéristiques de Fog et Cloud Computing.	23
3.2	Tableau comparatif entre le Fog et le Edge Computing.	24
4.1	Les différentes architectures spécifiques aux applications pour les systèmes de Fog Computing	32
4.2	Tableau comparatif entre des différents simulateurs de Fog.	36
5.1	Configuration des appareils dans la topologie.	46
5.2	Les résultats de la simulation, visant à évaluer la latence de l'architecture de parking proposée dans les environnements Fog et Cloud.	52
5.3	Les résultats de la simulation, visant à évaluer l'utilisation du réseau de l'architecture de parking proposée dans les environnements Fog et Cloud.	53
5.4	Les résultats de la simulation, visant à évaluer le temps d'exécution de l'architecture de parking proposée dans les environnements Fog et Cloud.	54
5.5	Les résultats de la simulation, visant à évaluer la consommation d'énergie de l'architecture de parking proposée dans les environnements Fog et Cloud.	56

INTRODUCTION

1

En raison du développement rapide de la population et de la croissance de l'industrie automobile, ce qui a entraîné une augmentation des véhicules de transport privés et pour obtenir un espace vide pour une personne. Dans la ville, le conducteur doit passer une moyenne de 6 à 14 minutes à chercher un parking, et dans les grandes villes, cette fois augmente à 18-20 minutes. On estime que cette fois la recherche d'une place de stationnement représente 30% de la congestion dans les rues des villes urbaines concentrées (comme les marchés, les zones touristiques ou les zones peuplées ...) ¹. Cela a également conduit à de nombreux problèmes : en perdant du temps, en pollution de l'environnement et, surtout, une consommation de carburant inutile. Pour aborder les cas susmentionnés, il a trouvé un nouveau terme appelé Smart Parking, et c'est une combinaison qui aide à gérer et à réduire la circulation sur les routes. Il fonctionne également pour gagner le temps nécessaire pour rechercher un lieu de stationnement en utilisant le cloud computing Infrastructure. Cependant, la latence est une préoccupation importante dans les applications basées sur le Cloud, y compris le transport intelligent et en particulier dans les systèmes de stationnement intelligents. Le Fog Computing amène les ressources de Cloud Computing à proximité immédiate de la périphérie du réseau, surmonte non seulement le problème de latence, mais fournit également des améliorations significatives. La principale motivation pour utiliser le calcul du brouillard dans ce travail est de minimiser la latence ainsi que l'utilisation du réseau dans le système global de stationnement intelligent. Afin d'évaluer et mesurer les métriques de ce travail nous avons utilisé iFogSim comme simulateur.

— Les deux premiers chapitres de ce mémoire seront consacrés à la présentation des concepts étudiés à savoir le Fog Computing, le Cloud Computing, le Edge Computing .

— La deuxième partie est consacrée à la présentation de l'état de l'art.

— La troisième partie quand à elle sera consacré au détail de notre contribution.

Enfin, nous concluons ce manuscrit et donnerons une liste de perspectives pour la continuité de ce travail.

1. <https://www.bablesmartcities.eu/fr/explorer/solutions/solution/smart-parking.html>

GÉNÉRALITÉS SUR LE CLOUD , EDGE ET IoT

2

SOMMAIRE

2.1	INTRODUCTION	3
2.2	LE CLOUD	3
2.2.1	Définition	3
2.2.2	Caractéristiques	4
2.2.3	Les modèles de services du Cloud	5
2.2.4	Les modèles de déploiement du Cloud	7
2.2.5	Les avantages et les inconvénients du Cloud	8
2.3	EDGE COMPUTING	10
2.3.1	Définition	10
2.3.2	Edge Computing : cas d'usage	10
2.3.3	Caractéristiques	10
2.3.4	Les avantages de l'Edge Computing	11
2.3.5	Internet des objets	12
2.3.6	Définition	12
2.3.7	Domaines d'applications de l'IoT	12
2.4	CONCLUSION	14

2.1 INTRODUCTION

En raison de la révolution technique, le monde a connu un développement des technologies de l'information et de la communication, qui a conduit à l'émergence du Cloud Computing, qui a pris de l'ampleur dans les milieux scientifiques et pratiques. Internet a également changé la façon dont les gens accèdent aux informations dont ils ont besoin. Et il a trouvé le terme Internet des objets qui connecte une myriade d'appareils, et pour mieux traiter cette énorme quantité d'informations, le terme Edge Computing a été trouvé, donc dans ce chapitre nous allons présenter des généralités relatives au Cloud, le Edg Computing et IoT

2.2 LE CLOUD

2.2.1 Définition

C'est le niveau le plus haut dans l'infrastructure. Il s'agit de serveurs puissants distribués dans des centres de données distants permettant d'héberger des applications destinées, par exemple, à l'analyse du Big Data et au stockage de données permanent [6]

Selon Wikipédia : le Cloud Computing il s'agit d'un concept de déportation sur des serveurs distants des traitements informatiques traditionnellement localisés sur le poste client.

Selon Cisco : le Cloud Computing est une plateforme de mutualisation informatique fournissant aux entreprises des services à la demande avec l'utilisation d'une infinité de ressources.

Le National Institute of Standards and Technology (NIST) a proposé deux définitions en 2009 :

« Le Cloud Computing est l'ensemble des disciplines, technologies et modèles commerciaux utilisés pour délivrer des capacités informatiques (logiciels, plateformes, matériels) comme un service à la demande ».

« Le Cloud Computing est un modèle d'accès à des ressources informatiques partagées et configurables (par exemple, réseaux, serveurs, stockage, applications et services), depuis un accès réseau, à la demande, de manière simple, à partir de quel type d'appareil et depuis n'importe quel endroit. Ces ressources sont disponibles rapidement et opérationnelles avec un minimum d'effort et d'interactions avec le fournisseur de service ».

Le Cloud Computing ou le Cloud, permet à ses utilisateurs de stocker les données, effectuer des tâches à l'aide de centres de données via Internet. Les ressources disponibles dans le Cloud accordées sont de faible puissance ou les terminaux aux ressources limitées effectuent des tâches complexes dans le Cloud, gain de temps de calcul exceptionnel [7].

2.2.2 Caractéristiques

Service à la demande

Le client peut s'allouer autant de ressources que nécessaire, et ceci à n'importe quel moment sans intervention avec les services du fournisseur mais toujours à sa demande [8].

Service mesuré (Pay As You Go)

Les utilisateurs payent ce qu'ils utilisent uniquement. Le service est donc mesuré par son utilisation.

Ceci peut varier selon le type de service, mais le principe reste toujours le même, par exemple :

- Un service de stockage peut mesurer l'espace alloué, sans pour autant facturer les autres ressources comme le processeur et la bande passante ;
- Un service de base de données peut mesurer la taille des tables, sans pour autant inclure le coût de leur création et leur traitement [8].

Élasticité

Les services Cloud peuvent être provisionnés et publiés de manière élastique, parfois automatiquement, afin que les clients puissent évoluer rapidement en fonction de la demande. Les capacités disponibles pour le provisionnement sont pratiquement illimitées. Les clients peuvent utiliser ces fonctionnalités à tout moment et en toute quantité. Les clients peuvent également faire évoluer l'utilisation, la capacité et les coûts du Cloud sans contrats ni frais supplémentaires. Avec une flexibilité rapide, vous n'aurez pas besoin d'acheter de matériel informatique. Au lieu de cela, peut utiliser les ressources de Cloud Computing du fournisseur de Cloud ¹.

Bande passante élevée et accès réseau universel

Les ressources mises à disposition des utilisateurs sont accessibles via Internet de n'importe quel endroit et en utilisant des plateformes hétérogènes (PC, téléphone, mobile, PDA, ordinateur portable, etc.) grâce à la capacité actuelle des réseaux [9].

Mise en commun des ressources ou partage des ressources (Pooling)

Les ressources mises à la disposition des utilisateurs en utilisant une plateforme de type Cloud sont accessibles via Internet quelque soit le périphérique utilisé (PC, Mac, tablette, SmartPhone, etc.). Ces ressources peuvent être partagées par plusieurs utilisateurs. Les fournisseurs s'appuient sur le concept de virtualisation, c'es-à-dire les ressources sont allouées et libérées selon les besoins des clients. Ceci a pour conséquence de masquer la localisation physique des

1. <https://www.synopsys.com/cloud/insights/essential-cloud-computing-characteristics.html>

ressources pour les présenter sous forme de machines virtuelles [9]. La figure 2.1 illustre les cinq caractéristiques du Cloud Computing.

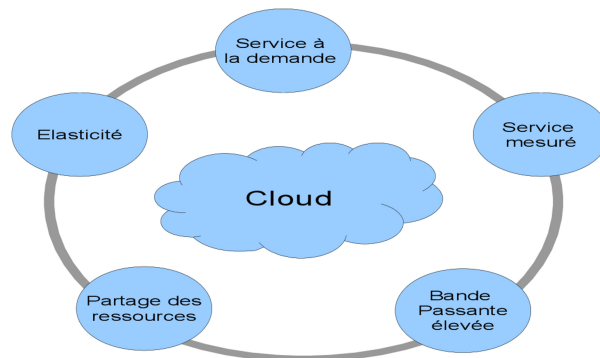


FIGURE 2.1 – Les cinq caractéristiques du Cloud Computing.

2.2.3 Les modèles de services du Cloud

Il existe les trois types de modèles de service Cloud (voir le figure 2.2) :

- 1-Infrastructure en tant que service (IaaS) ;
- 2-Plate-forme en tant que service (PaaS) ;
- 3-Logiciel en tant que service (SaaS).

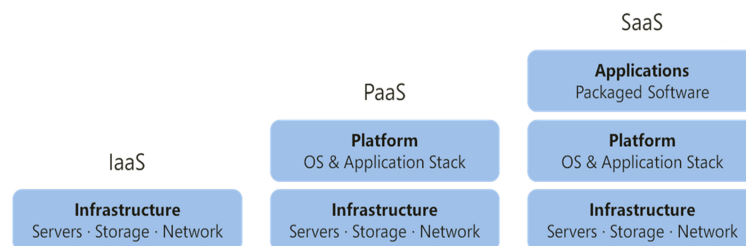


FIGURE 2.2 – SPI—Modèle d'offre de services du cloud.

Infrastructure en tant que service (IaaS)

IaaS est également connu sous le nom de Hardware as a Service (HaaS). Il s'agit d'une infrastructure informatique gérée sur Internet. Le principal avantage de l'utilisation d'IaaS est qu'il aide les utilisateurs à éviter le coût et la complexité de l'achat et de la gestion des serveurs physiques².

2. <https://cited.gr/en/what-is-spi-saas-paas-iaas/>

Caractéristiques de l'IaaS

Les caractéristiques d'IaaS sont les :

- Les ressources sont disponibles en tant que service ;
- Les services sont hautement évolutifs ;
- Dynamique et flexible ;
- Accès basé sur l'interface graphique et l'API ;
- Tâches administratives automatisées .

-**Exemple** : DigitalOcean, Linode, Amazon Web Services (AWS), Microsoft Azure, Google Compute Engine (GCE), Rackspace et Cisco Metacloud 2°.

Plate-forme en tant que service (PaaS)

La plate-forme de cloud computing PaaS est créée pour que le programmeur développe, teste, exécute et gère les applications.

Caractéristiques du PaaS

Les caractéristiques de PaaS :

- Accessible à différents utilisateurs via la même application de développement ;
 - S'intègre aux services Web et aux bases de données.
 - S'appuie sur la technologie de virtualisation, de sorte que les ressources peuvent facilement être augmentées ou réduites selon les besoins de l'organisation ;
 - Prise en charge de plusieurs langages et frameworks ;
 - Fournit une capacité de "mise à l'échelle automatique" .
- Exemple** : AWS Elastic Beanstalk, Windows Azure, Heroku, Force.com, Google App Engine, Apache Stratos, Magento Commerce Cloud et OpenShift 2°.

Logiciel en tant que service (SaaS)

SaaS est également connu sous le nom de "Logiciel à la demande". Il s'agit d'un logiciel dans lequel les applications sont hébergées par un fournisseur de services Cloud. Les utilisateurs peuvent accéder à ces applications à l'aide d'une connexion Internet et d'un navigateur Web 2°.

Caractéristiques du SaaS

Les caractéristiques de l'SaaS sont les :

- Géré depuis un emplacement central ;
- Hébergé sur un serveur distant ;
- Accessible par internet ;
- Les utilisateurs ne sont pas responsables des mises à jour matérielles et logicielles. Les mises à jour sont appliquées automatiquement ;
- Les services sont achetés sur la base du paiement à l'utilisation ;

-**Exemple** : BigCommerce, Google Apps, Salesforce, Dropbox, ZenDesk, Cisco WebEx, ZenDesk, Slack et GoToMeeting 2^o.

Différence entre IaaS, PaaS et SaaS

Le tableau ci-dessous illustre la différence entre les différentes infrastructures

IaaS	PaaS	SaaS
Utilisé par les Experts techniques	Utilisé par les développeurs	Pour tout les utilisateurs
Infrastructure.	Infrastructure+Plateforme.	Infrastructure + Plateforme + Logiciel
Fournit un centre de données virtuel pour stocker des informations et créer des plates-formes	Fournit des plates-formes pour créer, tester et déployer des applications.	Fournit des logiciels et des applications Web

TABLE 2.1 – La différence entre les services Cloud

2.2.4 Les modèles de déploiement du Cloud

- Public Cloud

L'infrastructure Cloud est accessible au grand public [10].

- Private Cloud

Le type de Cloud, disponible uniquement pour une seule organisation [10].

- Community Cloud

Dans ce type de modèle de déploiement Cloud, l'infrastructure du Cloud est partagé par plusieurs organisations et soutient une communauté spécifique ayant des préoccupations communes [10].

- Cloud hybride

Il s'agit d'une infrastructure Cloud qui est une composition de deux ou plusieurs Clouds privé, communautaire ou public [10].

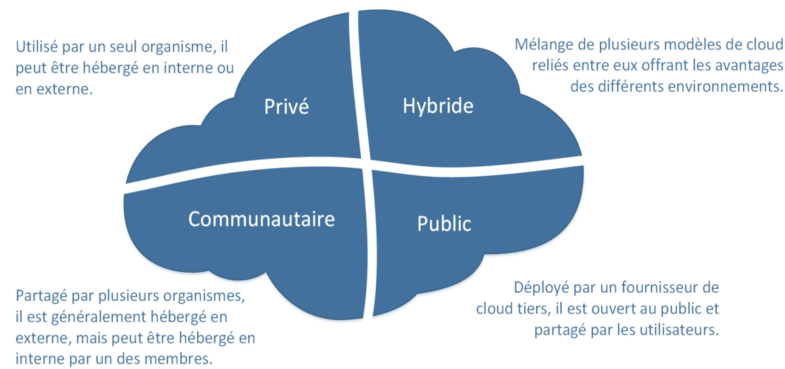


FIGURE 2.3 – Les Modèles de déploiement du cloud .

2.2.5 Les avantages et les inconvénients du Cloud

permi les avantages et les inconvénients du Cloud, nous citons

Avantages :

Nous pouvons résumer le succès du Cloud dans les points suivants [11] :

- La flexibilité et l'accès aux ressources de grandes échelles ;
- L'évolutivité ;
- L'agilité et la rapidité de mise en service, développement des produits ;
- Les données qui sont distribuées et répliquées sur de grandes distances géographiques ;
- Aucun investissement initial (Sans infrastructure propriétaire) ;
- Facturer à l'usage (Pay As You Go) ;
- Simplicité d'accès et d'utilisation de services complexes ;
- Réduction des coûts et du sous exploitation du matériel, licences, par consolidation et facturation réelle ;
- Puissance de calcul élastique ;
- Transfert de risques vers le fournisseur.

Inconvénients :

Certaines inconvénients présentes par le Cloud Computing³ .

- La connexion Internet

Le Cloud utilisant de manière intensive le transfert de données, il faut avoir une connexion très performante.

3. <https://missarte.wordpress.com/les-inconvénients-du-cloud-computing>

- Le coût du Cloud

Beaucoup d'entreprises ne regardent que les frais de stockage, mais il faut également prendre en compte les frais de transferts, qui peuvent s'avérer être importants, selon l'utilisation que l'entreprise fait du Cloud.

- L'optimisation des applications

Malgré une connexion internet rapide, avec un débit garanti, certaines applications Web peuvent s'avérer être très lentes. Elles peuvent s'avérer être plus limitées que des applications fonctionnant sur les propres ordinateurs de l'entreprise.

- La sécurité du Cloud

Plusieurs points sont à étudier :

– La sécurité vis-à-vis du stockage :

Si les données sont conservées dans un seul disque, ou si elles sont entre plusieurs unités de stockage.

– La sécurité et la confidentialité des données

Si le fournisseur de service assure des tests portant sur sa sécurité informatique et si de ses tests sont faits de façon régulières.

– La sécurité des locaux

Sont-ils inaccessibles pour des personnes malintentionnées

Le piratage

Certaines applications comme Facebook et Twitter sont très sujets aux attaques. Le piratage d'un compte d'entreprise pourrait avoir des conséquences néfastes pour la réputation de l'entreprise, tandis que l'utilisation imprudente des applications par un salarié pourrait offrir aux cybercriminels l'opportunité d'entrer dans le réseau et de soustraire des données des clients.

La plateforme

Il faut vérifier que l'hébergeur est capable de supporter de multiples plateformes car si ce n'est pas l'entreprise devra gérer plusieurs clouds ce qui peut s'avérer vite très complexe.

Les conditions de service

Il faut vérifier que les conditions de services sont conformes aux exigences de l'entreprise. Vu que ces contrats sont très détaillés, mieux faut lire tout en détail avant de prendre une décision. Le Cloud Computing n'est une mauvaise chose, bien au contraire, mais il est nécessaire avant de le mettre en place de peser le pour et le contre. En effet, certaines PME, de par leur activité, leur localisation, mais également par leurs applications utilisées, pourraient y percevoir plus de désavantages que de bénéfices.

2.3 EDGE COMPUTING

2.3.1 Définition

L'Edge Computing désigne une architecture informatique distribuée qui se caractérise par une puissance de traitement décentralisée. Concrètement, l'Edge Computing permet de traiter des données de façon directe par le périphérique qui les produit, ou par un ordinateur local. Il n'est dans ce cas plus nécessaire de transmettre les données à un datacenter distant pour les analyser. L'IDC (International Data Corporation) définit de son côté l'Edge Computing comme un réseau maillé de Micro Datacenters capables de traiter ou de stocker des données localement. On retrouve cette technologie principalement dans le domaine de l'IoT, où elle vient concurrencer le Cloud Computing.

L'Edge Computing se traduit en français par «traitement des données à la périphérie du réseau». Le terme, principalement utilisé dans l'IoT, désigne ainsi l'analyse des données au plus près de l'objet⁴.

2.3.2 Edge Computing : cas d'usage

Les principaux usages de l'edge computing dans l'IoT concernent la maintenance prédictive et le développement des véhicules autonomes. La fabrication industrielle dans l'industrie 4.0 bénéficie également de l'Edge Computing car la technologie permet de réduire la latence du traitement de l'information, en faisant les opérations en local et non sur un Data-Center⁴.

2.3.3 Caractéristiques

Sans limiter les fonctionnalités du Cloud Computing, Edge Computing doit avoir les exigences suivantes, dont certaines sont également définies pour le Cloud Computing⁵ : L'une des caractéristiques de l'Edge Computing est sa capacité à rapprocher les informations traitées de la source des données pour que le traitement des données proprement dit soit effectué rapidement et en toute sécurité, aussi près que possible de l'endroit où se trouve l'utilisateur émetteur

4. <https://www.journaldunet.fr/web-tech/dictionnaire-de-l-iot/1440664-edge-computing-definition-et-cas-d-usage-de-la-technologie>

5. <https://www.ip-systemes.com/details-qu+est+ce+que+l+edge+computing+definition+caracteristiques+avantages>
819.html

ou la source de ces données, et d'améliorer et optimiser l'utilisation des appareils électroniques capables de se connecter à l'internet. De cette façon, la rapidité de réaction sera plus grande.

L'Edge Computing est généralement considéré comme une assistance à distance. C'est-à-dire qu'elle ne nécessite pas de personnel technique résident spécialisé. En cas d'éventualité ou d'erreur du système, sa propre infrastructure est conçue de manière à pouvoir être réparée rapidement et facilement par le personnel local avec peu ou pas d'expertise technique. L'Edge Computing est également dotée, si nécessaire, d'un très petit nombre de personnel spécialisé qui gère le site de manière centralisée à distance. La latence et la réduction des coûts sont des caractéristiques essentielles de l'Edge Computing. La périphérie est l'emplacement le plus proche de l'abonné et où les données sont traitées ou stockées sans être transférées vers un emplacement central. Cela permet un meilleur contrôle des informations, car le traitement est effectué à proximité de la source des données. Elle s'appuie sur un service ou une application spécifique pour optimiser les coûts, les performances et l'expérience des utilisateurs.

L'Edge Computing est principalement utilisée dans les entreprises qui ont besoin d'une réponse plus rapide. En d'autres termes, la latence, qui définit le temps nécessaire à la collecte et au traitement des données sur un réseau, doit être réduite. Cela est possible en traitant les données au point de collecte, plutôt que d'avoir à attendre le traitement sur une plateforme centrale comme c'est le cas dans le Cloud. Les entreprises bénéficient ainsi de plus grands avantages.

2.3.4 Les avantages de l'Edge Computing

La technologie d'edge computing est souvent opposée à celle du Cloud Computing comme meilleure solution de traitement et de stockage des données. Elle présente certains avantages, notamment⁶ :

- La rapidité et les coûts du service de transmission des données sont grandement favorisés par le fait que le traitement des données est effectué à l'endroit même où elles sont générées. Comme nous l'avons mentionné plus haut, cela permet de répondre beaucoup plus rapidement à l'utilisateur. En outre, bien que le coût de l'Edge Computing soit plus élevé au moment de l'achat, il permettra à long terme de réaliser des économies grâce au modèle de traitement des données.

- Grâce aux systèmes de stockage de données distribués intelligents, il est possible d'analyser de grands ensembles de données IoT sans avoir à les envoyer dans la salle des serveurs. Cela accélère le processus de décision et vous permet de réduire les coûts. Très probablement, même la prolifération du réseau 5G ne signifiera pas que les données collectées par les voitures, les drones ou les avions autonomes seront envoyées dans la salle des serveurs. Au contraire, elle permet de rationaliser le traitement des données.

- Une meilleure gestion de la vie privée et des données, car actuellement, les données collectées par les installations IoT sont assez vulnérables au vol ou à la falsification. Mais la prise de conscience de leur existence finira par s'accroître et

6. <https://www.redhat.com/en/topics/edge-computing/what-is-edge-computing/companies-using-edge-computing>

ces menaces affecteront l'étape de la transmission des données vers les salles de serveurs sur les réseaux publics. Dans ce scénario, l'Edge Computing assurera une plus grande sécurité des données et empêchera qu'une seule panne ne fasse tomber l'ensemble du réseau.

- De réduire la bande-passante utilisée et le temps de latence du traitement de l'information car les flux de données sont traités en temps réel [12];
- D'assurer un meilleur niveau de sécurité puisque les données demeurent stockées à proximité de leur source et n'ont plus à traverser un réseau pour rejoindre un datacenter public [12];
- De réaliser des gains économiques importants [12];

2.3.5 Internet des objets

2.3.6 Définition

L'IoT est l'acronyme de Internet Of Things (Internet des Objets en français). Le terme IoT est apparu la première fois en 1999 dans un discours de Kevin ASHTON, un ingénieur britannique. Il servait à désigner un système où les objets physiques sont connectés à Internet. Il s'agit également de systèmes capables de créer et transmettre des données afin de créer de la valeur pour ses utilisateurs à travers divers services (agrégation, analytique, etc)

Selon l'UIT (Union Internationale des Télécommunications), l'Internet des Objets est défini comme « une infrastructure mondiale pour la société de l'information, qui permet de disposer de services évolués en interconnectant des objets (physique ou virtuels) grâce aux technologies de l'information et de la communication interopérables existantes ou en évolution ».

Au fil du temps, le terme a évolué et il englobe maintenant tout l'écosystème des objets connectés. Cet écosystème englobe, des fabricants de capteurs, des éditeurs de logiciels, des opérateurs historiques ou nouveaux sur le marché, des intégrateurs, etc . Cet éclectisme en fait sa richesse⁷.

L'IoT offre de grandes opportunités de marché pour les fabricants d'équipements, les fournisseurs de services Internet et les développeurs d'applications. Selon une analyse faite par Cisco, le nombre d'objets connectés à dépasser les 50 milliards en 2020 offrant un marché mondial de 2.700 à 6.200 milliards de dollars annuellement [1].

Quant aux consommateurs, les technologies IoT offriront une gamme impressionnante de nouveaux appareils et services, options qui faciliteront leur vie et leur travail [2] . La Figure 2.4 montre l'impact économique potentiel des applications de l'IoT [3].

2.3.7 Domaines d'applications de l'IoT

Les domaines d'application de l'IoT sont classifiés selon les auteurs de la référence [4] comme suit :

7. <https://www.digora.com/fr/blog/quest-ce-que-liot-et-pourquoi-mener-une-strategie-diot>

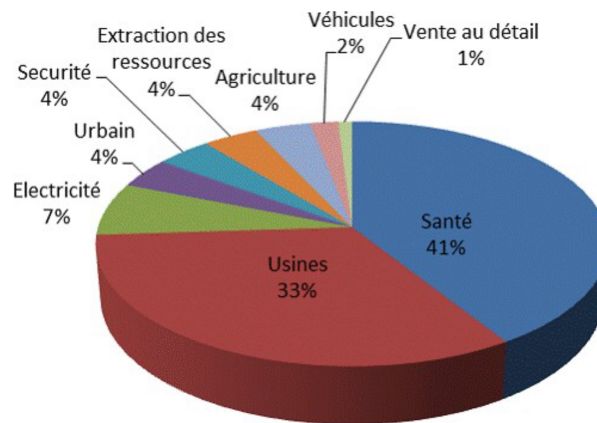


FIGURE 2.4 – L'impact économique potentiel des applications de l'IoT

— Transport et logistiques

Les voitures, les bus, les taxis ainsi que les trains sont de plus en plus équipés de capteurs, d'actionneurs et de puissance de traitement. Les routes elles-mêmes et les marchandises transportées sont également équipées par des étiquettes et des capteurs qui envoient des informations aux sites de contrôle et aux véhicules de transport pour mieux acheminer le trafic, aider à la gestion des dépôts, et fournir les marchandises transportées. L'une des applications réussies de l'IoT dans le secteur des transports est Waze, une application mobile de navigation GPS en temps réel. Cette application offre à ses utilisateurs des informations sur l'état du trafic en temps réel. Les utilisateurs, à leur tour, peuvent modifier la carte (sens de circulation, nouvelles rues, bouchons, etc.) afin de l'améliorer ou de suivre l'évolution du réseau routier. Actuellement, le nombre d'utilisateurs de Waze dépasse les 100 millions dans le monde. Une autre application de transport utilisant l'IoT est celle de Skydrone ; une entreprise française qui offre à ses clients

La possibilité de livrer leurs objets par drones autonomes. Les drones disposent d'une capacité de poids de charge (variable selon le modèle du drone) et ils sont équipés d'un système de détection d'obstacles aériens leur permettant d'être autonomes. Cette solution de transport aérien permet des livraisons rapides et flexibles.

— Télé-médecines

La télé-médecine est une forme de pratique médicale à distance utilisant des technologies de l'information et de la communication. Elle permet d'établir un diagnostic ou d'assurer un suivi pour un patient à distance. Par exemple, dans un système de santé à distance, la concentration quotidienne en oxygène, la glycémie et la pression artérielle d'un patient sont collectées automatiquement par des capteurs et ensuite transmises aux organismes de suivi. Une autre application de télé-médecine consiste à offrir des soins rapides pour les personnes ayant subi un accident ou un problème de santé. Pour cette dernière, Google propose une

solution à base de drones pour porter secours dans diverses situations d'urgence. Ces drones livrent un kit de premiers secours, des outils de diagnostic ou des médicaments [4].

— Environnements intelligents

Les maisons, les villes et les usines sont des exemples majeurs d'environnements intelligents. Les applications de ce domaine consistent à collecter et traiter des données de l'environnement pour réaliser des économies, sécuriser ou améliorer la vie des citoyens. Par exemple, dans le cas des maisons et des bureaux intelligents, le chauffage peut être adapté selon les conditions de la météo ; l'éclairage de la pièce peut être changé en fonction de l'heure de la journée ; les incidents peuvent être évités avec des systèmes de surveillance et d'alarme appropriés ; et l'énergie peut être économisée en éteignant automatiquement les équipements électriques non nécessaires . Dans le cas de l'industrie, des usines peuvent être enrichies par des capteurs et des actionneurs pour contrôler la chaîne de production, trier les produits défectueux ou arrêter le processus de production [4].

— Relation sociale

Les applications relevant de ce domaine sont celles qui permettent à l'utilisateur d'interagir avec d'autres personnes afin de maintenir et de développer des relations sociales. En effet, les objets peuvent automatiquement déclencher la transmission de messages à des amis pour leur permettre de savoir ce que la personne fait ou ce qu'elle a fait dans le passé. Quelques applications dans ce domaine sont : de partager des avis sur des produits, des services ou des endroits [4].

— Applications du futur

Les applications décrites dans les sections précédentes sont déjà déployées. En dehors de celles-là, nous pouvons envisager de nombreuses autres applications futures. Parmi ces applications on peut noter : l'œil intelligent, Cette technologie est très similaire aux lunettes de Google. Elle consiste à équiper les lentilles de contact avec des capteurs et des capacités de connectivité comme le Wi-Fi et le Bluetooth pour fournir de nombreuses fonctions telles que parcourir des cartes, lire des emails ou des messages, naviguer sur Internet ou capturer des vidéos [4].

2.4 CONCLUSION

Avec le Cloud Computing, une énorme quantité de données est transmise et traitée. Cependant il existe des problèmes tels que les retards dans le temps de réponse du service. De plus, au fur et à mesure que l'Internet des objets se développe, les domaines d'application des réseaux de capteurs s'élargissent et les demandes de traitement, de transmission et de sécurité en temps réel des informations contrôlant les appareils connectés augmentent. Ces dernières années,

les concepts du Fog Computing et d'Edge Computing ont été proposés pour atténuer ces limites en traitant des informations collectées à partir d'un appareil avant de les envoyer dans le Cloud pour réduire la quantité de transmission ou de s'exécuter sans recevoir d'informations de contrôle d'appareil depuis le Cloud. Ainsi, le retard de transmission de données dans l'appareil est réduit et l'appareil peut être contrôlé en temps réel. Compte tenu des prévisions de croissance et de développement de l'IoT, nous pensons que le plein potentiel de l'IoT ne peut, dans de nombreux cas, être débloqué qu'en combinant le Cloud, le Fog et le Edge. Dans le chapitre suivant nous allons apporter plus de détails sur le Fog Computing

FOG COMPUTING

3

SOMMAIRE

3.1	INTRODUCTION	17
3.2	DÉFINITION	17
3.3	CARACTÉRISTIQUES	17
3.4	LES AVANTAGES	19
3.5	LES INCONVÉNIENTS	19
3.6	COMPARAISON ENTRE LE CLOUD COMPUTING ET LE FOG COMPUTING .	22
3.7	COMPARAISON ENTRE LE FOG COMPUTING ET LE EDGE COMPUTING . .	23
3.8	CONCLUSION	25

3.1 INTRODUCTION

Plusieurs organisations ont utilisé le Cloud Computing. Au cours des dernières années, le Fog Computing a gagné en popularité. Le Fog Computing a été considérée comme une plate-forme qui étend les services de Cloud Computing. Ces services sont étendus à la périphérie du réseau. Fog computing fait également son travail à la périphérie du réseau. De cette façon, il a amélioré la qualité du service. Dans ce chapitre nous allons présenter une définition sur le Fog Computing et aussi nous discuterons plusieurs points essentiels liés au Fog, tels que : ses caractéristiques, ses avantages et ses inconvénients.

3.2 DÉFINITION

Le Fog Computing est une extension du paradigme Cloud dans le domaine des technologies de l'information, offrant des services plus limités mais en étroite proximité avec les appareils IoT. Ces dispositifs sont appelés nœuds de Fog.

Du point de vue de Cisco, le Fog Computing est perçu comme une extension du paradigme de l'informatique en nuage depuis l'épine dorsale du réseau. C'est une plateforme hautement virtualisée qui offre des services de calcul, de stockage et de mise en réseau entre les appareils finaux et les serveurs en nuage traditionnels. De nombreux chercheurs ont défini le Fog Computing sous de multiples angles.

Les auteurs de la référence [13] ont défini le Fog Computing comme suit : "Fog Computing est une architecture informatique géographiquement distribuée qui dispose d'un réservoir de ressources qui consiste en un ou plusieurs dispositifs hétérogènes connectés (y compris bord dispositifs) à la périphérie du réseau et non exclusivement soutenu par des services Cloud, fournir en collaboration des calculs élastiques, stockage et de communication (et de nombreux autres nouveaux services et tâches) dans des environnements isolés à grande échelle et des clients à proximité".

La publication spéciale 800-191 [16] du NIST l'a défini comme un paradigme horizontal, physique ou virtuel qui réside entre les dispositifs intelligents (généralement au sein des organisations) et l'informatique en nuage traditionnelle ou les centres de données sont connectés.

Selon le consortium OpenFog [15], Fog Computing est «Une architecture horizontale au niveau du système qui distribue des ressources et des services d'informatique, de stockage, de contrôle et de mise en réseau dans tout le continuum du Cloud aux Objets» Il s'agit d'une plateforme hautement virtualisée qui fournit des services de calcul, de stockage et de mise en réseau entre les appareils finaux et les centres de données d'informatique en nuage traditionnels [14, 17].

3.3 CARACTÉRISTIQUES

Les caractéristiques de Fog Computing sont les suivantes :

- Faible latence : en raison de la proximité des nœuds de Fog avec les dispositifs terminaux sur place, ce qui accélère considérablement la réponse et l'analyse.

- Soutien riche et hétérogène de l'utilisateur final : en raison de la proximité des dispositifs Edge aux nœuds de calcul ;
- Multi-emplacement dans un environnement contrôlé avec une plate-forme distribuée fortement virtualisée ;
- Améliorer la mobilité grâce à une communication plus directe entre les applications Fog et les dispositifs mobiles ;
- Interaction en temps réel, plutôt que traitement par lots, par exemple, pour les applications en nuage ;
- Connaissances contextuelles : comme ce fut le cas pour les dispositifs et les nœuds environnementaux ; connaissance et compréhension de l'environnement ;
- La distribution géographique, étant donné que l'environnement de Fog est réparti géographiquement, elle participe activement à la prestation de services de diffusion en continu de haute qualité ;
- Réseau d'accès sans fil : convient davantage à la détection sans fil des appareils qui nécessitent une analyse et une communication à temps partagé ;
- Soutien de l'hétérogénéité : étant donné que les nœuds de fog ont des facteurs de forme différents ; ils sont déployés dans une variété d'environnements répartis ;
- Interopérabilité transparente et meilleure coordination pour une meilleure communication entre les dispositifs de différents fournisseurs et domaines ;
- L'analyse en temps réel, qui est facile à réaliser en raison de l'ingestion et du traitement des données à proximité des sources ;
- Prise en charge d'un grand nombre d'applications industrielles grâce au traitement et à l'analyse en temps réel.

Ses caractéristiques sont utiles lors de la conception d'application Fog Computing ou villes intelligentes, maison intelligente, réseaux des applications éminentes comme les systèmes d'intervention d'urgence pour la surveillance et le rétablissement des inondations. La présence physique et l'accès ; et la mobilité qui distingue l'informatique en Fog des caractéristiques de l'informatique en Cloud et en périphérie [18].

De plus, le paradigme plus vaste du Fog consiste principalement en deux autres technologies connexes [12] :

- Cloudlets : il s'agit d'applications situées en périphérie du réseau , principalement pour résoudre le problème de la faible latence des communications entre les machines. Un cloudlet est un nœud de brouillard, un ordinateur ou une grappe d'ordinateurs riches en ressources (un centre de données dans une boîte), bien connecté à Internet et disponible pour être utilisé par des utilisateurs localisés. Les nœuds de brouillard peuvent être des éléments physiques ou virtuels et sont étroitement couplés avec les dispositifs finaux intelligents ou les réseaux d'accès. Ils ont quatre attributs principaux : l'autogestion, la puissance informatique, la latence de bout en bout et l'utilisation de techniques liées au nuage.

Dans l'architecture du réseau, les Cloudlets se trouvent au milieu de la structure en trois parties appareil mobile-Cloudlet-Cloud structure.

- Mobile Edge computing (MEC) : Il s'agit d'une technique liée à la mise en réseau mobile, au sein du réseau d'accès radio (RAN) et à proximité immédiate

des abonnés mobiles [19]. Il s'agit d'une architecture de réseau qui permet des et un environnement de services informatiques à la périphérie du réseau cellulaire. L'idée de base est que le traitement associé est plus proche du client cellulaire, ce qui réduit la congestion du réseau et augmente le rendement des applications.

3.4 LES AVANTAGES

Concernant les avantages de Fog , un examen plus approfondi du modèle de brouillard donne à penser qu'il s'agit de prendre des décisions aussi près que possible des données et des sources de production de données. Dans ce contexte, Hadoop ¹. et d'autres solutions de mégadonnées ont déjà commencé à rapprocher le traitement de localisation des données. La buée, c'est faire à plus grande échelle. Il y a des avantages évidents à prendre des décisions à proximité de l'endroit où les données sont générées et à ne pas avoir besoin que certaines données soient traitées dans le Cloud. Cela permet de résoudre les problèmes de sécurité et de protection de la vie privée, car les données précieuses et sensibles ne devraient pas vraiment se déplacer vers et via les réseaux d'informatique en nuage. Voici quelques avantages du Fog Computing :

- Une distribution géographique dense et un soutien en matière de mobilité ;
- Faible latence, connaissance de l'emplacement et amélioration de la qualité de service ;
- Plus grande agilité commerciale et réduction des coûts d'exploitation ;

3.5 LES INCONVÉNIENTS

Bien que la promesse du paradigme du Fog soit attrayante, il est important de noter et de comprendre les différents problèmes qui accompagnent l'utilisation et le déploiement de l'approche de calcul du Fog (Fog Computing).

Outre les problèmes hérités de l'informatique en Cloud, certains des problèmes spécifiques au Fog se rapportent à ce qui suit :

Défis dans l'appareil et le réseau

Les divers défis liés à l'appareil et le réseau sont le cadre décentralisé, les ressources de mise en réseau, l'hétérogénéité de l'appareil qui sont discutés comme suit.

1. Cadre décentralisé

Le cadre du Fog computing est décentralisé ce qui conduit à une structure redondante. Le même code est répété sur les dispositifs au bord du réseau . Il s'ensuit que le Fog devrait se concentrer sur la réduction de la redondance dans l'architecture décentralisée.

¹. Hadoop est un framework Open Source dédié au stockage et au traitement du Big Data

2. Ressources de réseautage

Dans l'architecture du Fog, les ressources réseau sont distribuées de manière aléatoire à la périphérie. De cette façon, la connectivité est plus complexe. Un réseau approprié qui a un Middleware pour maintenir un ensemble commun de ressources à la périphérie peut-être gérée pour permettre les ressources à l'application requise.

3. Hétérogénéité de l'instrument

L'architecture du Fog présente des extrémités hétérogènes. La nature de l'hétérogénéité a fait que la structure est plus variée. Les applications élaborées au moyen du Fog devraient aborder cet aspect de l'hétérogénéité des dispositifs et des réseaux.

Défis informatiques

1. Calcul à différents niveaux

Le système de Fog devrait toujours interagir avec les serveurs Cloud. L'objectif principal du système de Fog devrait être de répondre aux utilisateurs dans une limite de temps particulier au niveau inférieur et également de transmettre les calculs requis au cloud qui prendra plus de temps. Peu de calculs sont transférés aux nuages qui ne sont pas limités à leur temps de réponse et à la puissance, tandis que d'autres parties du calcul seront exécutées près du bord avec un faible coût de ces dernières. Il est difficile de déterminer quels calculs doivent être effectués au niveau du Cloud et du Fog.

2. Distribuer les ressources de calcul

Le calcul de la limite ne peut pas comporter les ressources requises. Ces ressources peuvent être acquises à partir de nœuds de Fog alternatifs. Ceci a augmenté l'exigence d'assigner des calculs de ressources entre les différents nœuds de Fog. Cela nécessite une approche pour combiner la mémoire, les calculs et les ressources réseau pour former un Pool commun. Diverses applications peuvent s'adresser au pool et réserver leurs ressources en fonction de leurs besoins. La présente recherche se concentre sur la nécessité de développer un pool commun contenant les ressources plutôt que les dispositifs à la périphérie de l'informatique.

a. Transférabilité : Les nœuds sur le bord (Edge) de l'environnement de Fog peuvent être portatifs qui soulève le besoin de calcul n'importe où, n'importe quel moment. Il faut donc procéder à un calcul omniprésent dans le fog. La portabilité est une limitation en architecture de brouillard (le Fog).

b. Calculs dans un environnement distribué Fog Computing est un domaine qui a distribué le calcul où l'exactitude doit être vérifiée. Les applications impliquant le cadre de Fog (le Framework de le Fog) doivent être développées de manière à réduire les écarts dans les calculs. Ces écarts devraient être prouvables.

Défi de mobilisation

Du point de vue de la mobilisation, OpenFog est spécifié comme environnement de niveau n. Néanmoins, la flambée extrême du nombre de couches de niveau de brouillard peut causer des problèmes tardifs dans le paradigme de fog récemment développé. Ainsi, le nombre de niveaux établis dans l'étude de cas doit être fermé. Les résultats de mobilisation seront acceptés lorsqu'ils seront établis sur des demandes comme le type de travail effectué par chaque niveau, l'ensemble des capteurs utilisés, la compétence des dispositifs antibrouillard et la fiabilité et la latence des dispositifs de Fog. Cependant, il est essentiel de déterminer de quelle façon ces demandes devraient être satisfaites.

1. Utilisation des ressources

L'utilisation des ressources est extrêmement dynamique et différente dans un environnement comme un fog pour la raison que les appareils sont de nature diverse et accessibles. Tous ces dispositifs de Fog sont responsables du fonctionnement de l'application eux-mêmes.

2. Difficulté de panne d'un dispositif Fog

La probabilité de panne des dispositifs Fog est en constante augmentation parce que les dispositifs Fog sont diffus et que le contrôle de ces dispositifs n'est pas centralisé. Par conséquent, les dispositifs de fog peuvent s'effondrer pour plusieurs raisons comme l'activité du consommateur final, la panne de matériel et le plantage de logiciels. En plus de ces questions, peu d'autres qui peuvent jouer un rôle majeur sont la source d'énergie, la connectivité et la flexibilité. Plusieurs dispositifs de Fog peuvent être reliés via Wi-Fi, il est compréhensible que la liaison Wi-Fi n'est toujours pas fiable. La majorité d'entre eux sont des appareils mobiles qui sont connectés sans fil, donc ces appareils pourraient modifier la position à d'autres grappes régulièrement.

3. Contestation du traitement des demandes

Le paradigme du Fog gère des millions de dispositifs IoT qui sont régulièrement des applications réactives et non réactives. Le niveau de service obtenu et la nature sont très différents dans le Fog. Par conséquent, l'obtention du service est un défi particulier pour le domaine de Fog.

Défi de complexité

Dans Fog Computing il y a plusieurs capteurs et dispositifs de l'IoT (ou bien IdO) qui sont développés par de nombreuses entreprises, le choix du mécanisme idéal devient difficile, principalement avec la structure matérielle diverse, configuration logicielle, et les demandes individuelles. De plus, dans de rares cas, les demandes d'application à sécurité maximale nécessitent des protocoles et des dispositifs explicites pour fonctionner, ce qui accentue le recul du processus.

Défis en matière de sécurité

L'architecture de Fog se compose de divers dispositifs hétérogènes. Ils peuvent être vulnérables à différentes attaques discuter de l'homme au milieu de l'attaque dans l'environnement de Fog. Les deux grands problèmes du Fog sont les données et le réseau. Il dépend également d'autres facteurs de centre de données de Cloud. Les dispositifs de fog sont déployés dans un environnement qui n'est pas très sécurisé, donc toute attaque vulnérable ou physique peut facilement être réalisée. Il faut utiliser les appareils de l'edge en toute sécurité dans le Fog.

3.6 COMPARAISON ENTRE LE CLOUD COMPUTING ET LE FOG COMPUTING

À la différence de l'architecture de Cloud centralisée, Fog computing est connu pour son architecture géographiquement distribuée, qui accélère la transmission de flux de données. Le Cloud est centralisé et contient une grande quantité de données qui peuvent être positionnées dans le monde entier, loin des appareils clients. Le Fog est disséminé et contient plusieurs petits noeuds situés à proximité des dispositifs du client. Le Fog est la couche entre le Cloud et les appareils comme les ordinateurs, les ordinateurs portatifs, les appareils mobiles, etc. Comme le Fog agit comme médiateur, le transfert des données prend moins de temps. En l'absence de couche, le Cloud doit communiquer directement avec les terminaux, ce qui prend plus de temps que l'utilisation de Fog computing. Le Cloud Computing a une faible latence, mais ne se compare pas au fog computing. Le Fog Computing a une faible latence en termes de réseau .

Caractéristiques	Cloud computing	Fog computing
Temps de réponse	Minutes, jours, semaines	Millisecondes, sous-secondes
But	Immensité et puissance provisionnement des administrations informatique	Améliorer les compétences et les performances du processus qui est transporté vers le cloud pour la manutention, la vérification et le stockage.
Latence	Élevé	Faible latence
Type des requêtes	Nombreuses petites allocations	De nombreuses affectations élevées
Unité d'allocation	Toutes formes et tailles (large et étroite)	Toutes formes et tailles (large et étroit)
Infrastructure	3 modèles (PaaS, IaaS, SaaS)	Flexible
Exemple du monde réel	Amazon Web Service (AWS), Applications Google	Les applications importantes de Fog impliquent des interactions en temps réel plutôt que le traitement par lots.
Prix du service	Prix des services publics réduits pour les gros clients	Le prix et le paiement des services publics sont établis en fonction des utilisations
Mobilité	Limitée	Supporte la mobilité
Connexion au dernier kilomètre	Ligne louée	Sans fil
Sécurité	Indéfini	Possible, déterminé
Supporter la gestion des utilisateurs	Centralisé ou délégué à une tierce partie	Centralisé

TABLE 3.1 – Tableau comparative entre les caractéristiques de Fog et Cloud Computing.

3.7 COMPARAISON ENTRE LE FOG COMPUTING ET LE EDGE COMPUTING

Bien que Fog et Edge Computing font tous deux références à l'intelligence, le traitement et le stockage à la périphérie du réseau, c'est à dire. plus proche des sources de données, la principale différence est de savoir exactement où l'intelligence et le traitement sont placés. Alors que Fog Computing pousse l'intelligence au niveau du réseau local de l'architecture réseau, le traitement des données dans un nœud Fog ou la passerelle IoT, Edge Computing place l'intelligence, les capacités de traitement et de communication d'une passerelle Edge directement dans les appareils intelligents comme les contrôleurs d'automatisation programmables (PAC) .

De plus, Edge Computing est une expression plus ancienne que le terme Fog Computing. Bien que Edge Computing soit généralement appelé l'endroit où les services sont instanciés, Fog Computing implique la distribution des ressources

de communication, de calcul et de stockage et des services sur ou à proximité des dispositifs et systèmes sous le contrôle des utilisateurs finaux [17].

Fog Computing	Edge computing
Les informations sont calculées à l'intérieur des nœuds de Fog ou de la passerelle de l'IoT qui se trouve à l'intérieur du réseau local	L'information est calculée sur le capteur ou l'appareil IoT, sans être déplacée vers les centres de données ou le Cloud.
Il peut gérer plus d'informations à un intervalle de temps particulier qui se rallie essentiellement sur les capacités de Edge grâce à sa capacité à gérer les besoins en temps réel	Il conserve toutes les informations et les calculs sur l'appareil qui l'a initialement généré.
Il peut calculer des volumes élevés d'informations avec un délai d'interruption très faible ou nul	L'information se trouve sur les « bords » (Edges) du réseau et peut être traitée instantanément.
Il ne nécessite pas un accès connecté continu. L'information peut être conservée localement ou établie à partir d'un stockage local	Les informations sont principalement calculées localement, puis envoyées aux centres de données centraux.

TABLE 3.2 – *Tableau comparatif entre le Fog et le Edge Computing.*

Exemples d'applications du Fog Computing

Le Fog Computing a d'énormes avantages dans les applications en temps réel. Il est largement utilisé dans l'IoT qui implique des données en temps réel. Il agit comme une version étendue de Cloud Computing. Il est un intermédiaire entre le Cloud et les utilisateurs finaux (plus proche des utilisateurs finaux). Il peut être utilisé dans les deux sens, qui peut être entre la machine et la machine ou entre l'homme et la machine.

1. Analyse des mégadonnées mobiles

Dans l'IdO (Internet des objets), les données sont collectées en vrac et ne peuvent pas être stockées dans le Cloud (pas assez efficaces). Dans de telles situations, il est bénéfique d'utiliser le fog computing au lieu de le cloud computing, car les nœuds de Fog sont beaucoup plus proches des systèmes finaux. Cela élimine également d'autres problèmes tels que le retard, le trafic, la vitesse de traitement, le temps de livraison, le temps de réponse, le transport des données de stockage et le traitement. Le brouillard pourrait être l'avenir des applications IdO.

2. Pression de l'eau dans les barrages

Les capteurs installés dans les barrages envoient des données au Cloud où les données sont analysées et les responsables sont alertés en cas de divergence. Le problème rencontré ici est le retard de l'information qui pourrait être dangereux. Pour résoudre ce problème, Fog est utilisé, et comme il est proche des systèmes finaux, il est plus facile de transmettre des données, de les analyser et de donner une rétroaction instantanée.

3. Service utilitaire intelligent

L'objectif principal est d'économiser du temps, de l'argent et de l'énergie. L'analyse des données doit être exécutée chaque minute pour rester à jour. Cela implique principalement les utilisateurs finaux, par conséquent, le Cloud pourrait ne pas servir l'objectif. Ces applications informent les utilisateurs tous les jours sur les appareils qui économisent le plus d'énergie. IoT crée également beaucoup de trafic dans le réseau où l'envoi d'autres données est difficile, donc le brouillard est une bonne alternative.

3.8 CONCLUSION

Bien que le Cloud Computing puisse aujourd'hui être considéré comme bien établi, les domaines d'application modernes tels que l'IdO, la conduite autonome ou même les applications mobiles essayant d'exploiter tout le potentiel des futurs réseaux 5G nécessitent une extension du Cloud vers la périphérie, conduisant ainsi naturellement au nouveau paradigme Fog Computing. Dans le chapitre suivant, nous allons présenter quelques travaux connexes sur les différentes architectures de Fog Computing proposées par différents auteurs

SOMMAIRE

4.1	INTRODUCTION	27
4.2	ARCHITECTURES DU SYSTÈME DE FOG	27
4.2.1	Architectures agnostiques pour les applications d'utilisateur final des systèmes Fog	27
4.2.2	Architectures spécifiques aux applications pour les systèmes de Fog	30
4.3	ALGORITHMES DE POSITIONNEMENT DES DONNÉES SÉLECTIONNÉES . .	33
4.3.1	Algorithme Edge-Ward	33
4.3.2	Algorithme Cloud-Only	33
4.3.3	Algorithme de Mapping	33
4.4	SIMULATEUR DE FOG COMPUTING	34
4.4.1	Simulateur IFogSim	34
4.4.2	Simulateur FogTorch	34
4.4.3	Simulateur FogNetSim++	34
4.4.4	Comparaison entre les différents simulateurs de Fog	35
4.5	CONCLUSION	36

4.1 INTRODUCTION

Au fil des ans, les paradigmes informatiques sont passés de l'informatique distribuée, parallèle et en grille à l'infonuagique. Le Cloud Computing , offre plusieurs capacités inhérentes, comme l'évolutivité, l'affectation des ressources à la demande, la réduction des efforts de gestion, un modèle de tarification souple (paiement à l'utilisation) et des applications et des services faciles à fournir. De nos jours, le Cloud Computing est largement utilisé. Cependant, il a encore quelques limites. La limitation fondamentale est la connectivité entre le Cloud et les terminaux.

Fog computing est un paradigme informatique introduit pour relever ces défis. Il est maintenant promu par le consortium OpenFog qui a récemment publié quelques livres blancs . Il s'agit d'une architecture nouvelle qui étend l'architecture de cloud computing traditionnelle à la périphérie du réseau. Dans le cas du fog, le traitement de certains composants d'application (p. ex., ceux sensibles à la latence) peut avoir lieu à la périphérie du réseau, tandis que d'autres (p. ex., les composants tolérants aux retards et à forte intensité de calcul) peuvent se produire dans le nuage. Dans ce chapitre, nous allons étudier les architectures du Fog Computing qui ont été proposées au cours des dernières années ainsi que l'outil de simulation et les algorithmes de positionnement des données sélectionnées existantes [37].

4.2 ARCHITECTURES DU SYSTÈME DE FOG

Comme c'est le cas pour les autres systèmes informatiques distribués à grande échelle, les architectures proposées à ce jour pour les systèmes Fog sont agnostiques ou spécifiques aux applications.

Les architectures agnostiques d'applications se concentrent sur des facettes architecturales spécifiques bien qu'elles ciblent des applications d'utilisateur final (User-End) en général.

D'autre part, la plupart des architectures spécifiques aux applications se concentrent sur la santé. Par ailleurs, des architectures sont proposées pour les véhicules connectés et la vie intelligente.

4.2.1 Architectures agnostiques pour les applications d'utilisateur final des systèmes Fog

a - Architectures d'approvisionnement des applications pour les utilisateurs finaux

Deux architectures de programmation sont proposées. On propose aussi une architecture plus générale. Le cycle de vie de l'application de l'utilisateur final consiste en trois phases principales : Développement, déploiement et gestion (y compris l'exécution) [20].

La première architecture de programmation est connue sous le nom de Brume Mobile, elle est proposée par Hong et al. [21]. Elle permet aux développeurs d'écrire des programmes pour des nœuds spécifiques dans l'IdO (utilisateurs

finiaux), le Fog et les couches de Cloud. Le déploiement peut se faire dans n'importe quelle couche et le même code peut être déployé sur plusieurs nœuds différents qui appartiennent à n'importe quelle couche.

La deuxième architecture de programmation traite du flux de données distribuées (DDF), proposée par Giang et al. [22]. L'essence est que la topologie d'application est exprimée comme un graphique dirigé (flux) composé de nœuds avec chaque nœud correspondant à un composant d'application.

Yangu et al. [36] proposent une architecture plus générale par rapport à Hong et al. [21] et Giang et al. [22] qui repose sur deux principes : concevoir l'architecture comme une extension au PaaS existant et utiliser le paradigme REST pour les interactions.

b - Architectures pour la gestion des ressources dans les systèmes Fog

Lorsque la couche de Cloud, la couche de Fog et la couche IdO(utilisateurs finaux) sont intégrées dans un système de Fog, la gestion des ressources devient un enjeu critique. La migration, l'affectation et l'ordonnancement des ressources sont les sujets abordés par les chercheurs jusqu'à présent.

Bittencourt et al. [23] ont travaillé sur la migration des ressources, en se concentrant sur la migration des machines virtuelles entre les nœuds de Fog. L'objectif est de garder la machine virtuelle disponible lorsque l'utilisateur se déplace. Ils supposent que la machine virtuelle contient les données et le(s) composant(s) d'application de l'utilisateur. La migration se fait de manière à ce que les utilisateurs ne remarquent aucune dégradation dans les performances de leur application.

Agarwal et al. [35] se concentrent sur l'allocation des ressources. L'architecture proposée implémente un algorithme qui répartit la charge de travail entre le Cloud et les couches de Fog. L'architecture proposée est basée sur des couches, composée de trois couches. Dans la couche IoT(utilisateurs finaux), les auteurs proposent la couche client, composée de clients, d'appareils mobiles et de capteurs. Dans la couche de Fog, les auteurs proposent la couche de Fog qui implique un module appelé gestionnaire de serveur de Fog. Ce module est chargé de vérifier si suffisamment de ressources de calcul sont disponibles pour les composants hôtes. Sur la base de cette disponibilité, il exécute tous les composants ou en exécute certains tout en reportant l'exécution d'autres, ou même envoie certains des composants au(x) nœud(s) de la couche Cloud.

Le gestionnaire de serveur de Fog est également responsable de la gestion du cycle de vie des machines virtuelles. Pour la strate de Cloud, les auteurs proposent la couche de cloud qui comprend les modules responsables de l'exécution des composants reçus du gestionnaire de serveur de Fog.

Cardellini et al. in [24] se concentrent sur la planification des tâches. Ils proposent une extension d'une architecture existante appelée Storm, afin d'exécuter un planificateur distribué sensible à la QoS (qualité de service). Le planificateur est responsable du déploiement des composants de l'application sur le Pool de ressources disponibles. Storm est un système de traitement des flux de données (DSP) open source. Cette extension permet à Storm de fonctionner dans des environnements géographiquement distribués et très dynamiques. Les auteurs ont

ajouté de nouveaux modules à l'architecture qui permettent d'exécuter un planificateur distribué sensible à la qualité de service de service. Ils ont également inclus des capacités d'adaptation à l'architecture.

Kapsalis et al. [25] présente une architecture pour une plate-forme compatible brouillard qui est responsable de l'allocation et de la gestion des ressources de calcul nécessaires pour héberger les composants de l'application. Il utilise une méthode de communication distribuée basée sur le modèle de publication/abonnement et le protocole MQTT (Message Queueing Telemetry Transport). L'architecture proposée comporte quatre couches. Dans la couche IoT/utilisateurs finaux, les auteurs proposent la couche de périphérique et la couche de concentrateur. La couche de dispositif se compose de dispositifs physiques contraints, tels que des capteurs et des actionneurs, et la couche de concentrateur se compose de passerelles. Les passerelles n'effectuent aucun calcul. Leur rôle est de convertir le protocole de communication des périphériques de la couche de périphérique et de l'envoyer sur MQTT (si nécessaire) aux couches supérieures. Ils agissent en tant que médiateurs. Dans la strate de Fog, Les auteurs proposent la couche de Fog composée de bords (Edges) stables et de bords (Edges) mobiles. Le bord (Edge) stable comprend un module architectural appelé Fog Broker qui est chargé d'enrichir les messages reçus des couches inférieures. Ils sont également responsables de la gestion et de l'attribution des tâches. Et le bord mobile est responsable de la communication avec les appareils de la couche inférieure. Enfin, pour la strate Cloud, les auteurs proposent la couche Cloud qui est responsable de l'analyse et du stockage à long terme.

c - Communication Architectures

Les communications intra-stratum (À l'intérieur des couches) et inter-stratum (entre les couches) sont requises dans un système de Fog. Des recherches ont déjà été menées sur les communications inter-stratum et intra-stratum. Shi et al. [26] traitent de la communication inter-stratum entre la strate IoT/utilisateurs finaux et la strate de brouillard. Plus précisément, ils utilisent le protocole d'application contrainte (CoAP) pour la communication.

CoAP est un protocole de transfert Web spécialisé utilisé pour les nœuds contraints dans l'IdO. L'objectif principal de leur travail est de permettre aux nœuds de la couche de Fog et aux périphériques de la couche IdO/utilisateurs finaux de partager les services et les capacités de manière transparente en utilisant le protocole REST et IoT CoAP. Pour la couche IdO/utilisateurs finaux, les auteurs ont considéré à la fois les capteurs à faible énergie et haut de gamme qui appartiennent à cette couche. Ces appareils agissent comme des clients CoAP.

Slabicki et al. [27] se concentrent sur la communication intra-strate entre les dispositifs dans la couche l'IdO/utilisateurs finaux Les auteurs examinent trois cas : la communication directe entre les appareils, la communication par le Fog et la communication par le Cloud.

Krishnan et al. [28] proposent une architecture qui permet à l'appareil utilisateur de décider si un composant doit être exécuté dans le Cloud ou dans les couches de Fog. Ils traitent de la communication inter-stratum entre le Fog et les couches de Cloud. Dans la couche l'IdO(utilisateurs finaux), les paquets sont

étiquetés avant d'être envoyés. Ces balises comprennent des informations sur la destination (c'est-à-dire le Cloud ou le Fog). Dans la couche de fog, lorsqu'un nœud de Fog reçoit un paquet, il le décode et l'envoie au nœud respectif pour traitement. Si le nœud est dans le Cloud, le nœud de Fog place l'adresse IP source comme celle du générateur de données et l'adresse de destination comme celle du Cloud. Il transmet ensuite le paquet à la couche du Cloud pour traitement.

Aazam et al. [29] traitent des communications inter-stratum. Cependant, contrairement à Shi et al. [26] qui se concentrent sur les communications entre la couche de l'IdO(utilisateurs finaux) et la couche de Fog, ils se concentrent sur la couche de Fog/les communications de la couche Cloud. Ils proposent une architecture qui tente de réduire le nombre de paquets envoyés au Cloud. Ceci est fait afin de réduire la charge sur le Cloud et d'atténuer les frais généraux de communication sur le Cloud.

Similaire à Aazam et al. [29], Moreno-Vozmediano et al. [30] traitent de la communication inter-stratum entre la couche de Fog /et la couche de Cloud. Ils ont proposé une architecture appelée Fog et Cloud hybrides (Hybrid Fog and Cloud (HFC)) d'interconnexion de l'infrastructure logicielle (Framework) pour permettre une configuration simple et efficace des réseaux virtuels pour interconnecter les domaines de Fog et de Cloud géographiquement distribués.

d - Architectures pour la fédération des Cloud et du Fog

Zhankieev [31] est le seul chercheur qui a jusqu'à présent abordé la question de la fédération en proposant une architecture appelée Plate-forme de visite dans le Cloud (Cloud Visitation Platform (CVP)). L'architecture proposée permet la fédération entre le Cloud et le Fog.

4.2.2 Architectures spécifiques aux applications pour les systèmes de Fog

Le tableau 4.1 exprime les principales caractéristiques et exigences pour les travaux proposant une architecture pour des systèmes de Fog spécifiques à des applications.

Architecture	Chercheur	Mesures de performance	Contribution majeure
Architectures pour les soins de santé	MasipBruin et al [41] .	- Battement cardiaque et volume d'oxygène par rapport à la mobilité - Ratio de taux d'incidence (IRR) pour le taux de réadmission et la mortalité respiratoire	- Proposer une architecture appelée F2C afin de soutenir les personnes touchées par la BPCO.
	Monteiro et al. [42]	-Céridoïde spectral - Taux de croisement zéro - Énergie à court-temps	- Proposer une architecture appelée FIT qui traite et analyse les données cliniques de la parole des patients atteints de maladie de Parkinson et de troubles de la parole.
	Dubey et al. [43]	- Réduction des données en pourcentage obtenue pour les scénarios de différence en effectuant le traitement sur le brouillard - Temps de traitement sur le brouillard pour différentes méthodes et algorithmes	- Proposer une architecture axée sur les services pour les applications de télésanté. Ils étudient un cas de troubles de la parole.
Architectures pour véhicules connectés	Hou et al. [44]		- Proposer une architecture pour les applications de véhicules appelée VFC. Les véhicules sont utilisés comme infrastructure de communication et de calcul.
	Datta et al. [45]		- Proposer une architecture pour les véhicules connectés, où le fog est co-situé aux RSU et aux passerelles M2M.
	Truong et al. [46]		- Proposez une architecture VANET qui tire parti du Fog computing et du SDN.

Architectures pour une vie intelligente et des villes intelligentes	Li et al. [47]	-Latence	- Proposez une architecture pour le Fog qui prend en charge l'application de la vie intelligente.
	Yan et al. [48]	- Comparaison du temps moyen de traitement des données compte tenu du traitement du fog et du Cloud	- Proposer une architecture pour les applications de réseau intelligent qui permet le stockage et le traitement des données.
	Tang et al. [49]	- Quantité de données transmises au Cloud - Délai de réponse	- Proposer une architecture hiérarchique distribuée pour soutenir l'intégration d'un grand nombre de composants d'infrastructure dans les villes intelligentes.
Architectures pour les autres applications	Gazis et P al. [50]	- Nombre de paquets - Utilisation du processeur	- Proposez une plateforme appelée (AOP), où le Fog est co-situé à la passerelle/routeur.
	Xu et al. [51]	- Flèche-fenêtre de congestion -Début	- Proposer une architecture pour l'analyse des données basée sur le SDN.
	Aazam et al. [52]	- Retard du téléchargement et de la synchronisation des données vers le cloud et le Fog	- Proposer une architecture pour l'alerte d'urgence et la gestion par le Fog.

TABLE 4.1 – Les différentes architectures spécifiques aux applications pour les systèmes de Fog Computing

4.3 ALGORITHMES DE POSITIONNEMENT DES DONNÉES SÉLECTIONNÉES

Pour évaluer le comportement de performance de différentes stratégies de placement de données, ce travail a pris en compte trois algorithmes différents. Les algorithmes de placement de données sélectionnés sont : Edge Ward, Cloud-only et Mapping.

4.3.1 Algorithme Edge-Ward

Edge-ward [33] est basé sur une stratégie de premier arrivé, premier servi (First Come-First Served (FCFS)), et permet donc de placer les données aussi près que possible du edge du réseau, sur les nœuds Fog. Cet algorithme a été sélectionné car il représente une stratégie de placement axée uniquement sur l'Edge.

Si un nœud de Fog spécifique ne peut pas répondre aux exigences d'application, Edge-ward sélectionne des dispositifs de fog supplémentaires. L'algorithme crée des tuples de périphériques représentant les chemins par lesquels les "modules d'application" (services) sont exécutés.

Les requêtes des applications sont traitées selon l'ordre dans lequel elles arrivent jusqu'à ce qu'il y ait des ressources informatiques disponibles à chaque niveau hiérarchique. Si le dispositif Fog sélectionné pour exécuter une application n'a pas de capacité de calcul disponible, l'algorithme recherche un dispositif Fog ayant une capacité au niveau de la couche supérieure de la hiérarchie topologique du réseau. L'algorithme a été choisi parce qu'il présente de meilleures performances lors de la planification des ressources (en particulier du processeur) des dispositifs de brouillard de manière hiérarchique. Si un dispositif de brouillard n'est pas en mesure de répondre aux exigences d'un module d'application, il peut être programmé dans le Cloud.

4.3.2 Algorithme Cloud-Only

Cet algorithme est basé sur l'implémentation traditionnelle exécutée dans le Cloud et utilise une stratégie de priorité de retard [33]. Le placement dans le Cloud uniquement suppose que tous les modules d'application s'exécutent dans des centres de données. Par conséquent, cet algorithme suit la situation traditionnelle où les capteurs capturent des données ; ces données sont traitées sur le Cloud ; le Cloud envoie des informations aux actionneurs si nécessaire.

4.3.3 Algorithme de Mapping

L'algorithme de Mapping repose sur une stratégie simultanée. Les requêtes d'application sont mappées de préférence aux dispositifs Fog, indépendamment de leurs capacités et exigences d'application [34]. Par conséquent, si la capacité du processeur du périphérique Fog sélectionné n'est pas suffisante pour répondre à une exigence d'application, alors Mapping forme une file d'attente de traitement sur le nœud Fog.

L'algorithme reçoit comme paramètre les listes de chemins possibles par lesquels l'application peut être exécutée à la suite d'une traversée de feuille à racine.

4.4 SIMULATEUR DE FOG COMPUTING

4.4.1 Simulateur IFogSim

Les algorithmes ont été évalués au moyen de simulations effectuées

Avec le simulateur iFogSim [43]. iFogSim permet la simulation des politiques de gestion des ressources et de planification des applications. Le simulateur intègre différents modules. Les appareils IoT modélisent des appareils au niveau du terrain et sont définis en fonction de paramètres spécifiques, tels que l'utilisation du processeur.

Les applications IoT construisent les éléments de traitement des données sur la base de modèles d'application spécifiques, qui sont des objets qui simulent des tuples entre les sources et les destinations. Pour le cas spécifique du scénario défini, chaque application correspond à un tuple de brouillard, c'est-à-dire une session établie pour transmettre des données entre les capteurs et le nuage, ou des capteurs et des dispositifs de brouillard (par exemple, passerelle IoT, smartphone). Chaque tuple est caractérisé par les attributs suivants :

- i) la longueur du processeur, en millions d'instructions par seconde (MIPS) .
- ii) la longueur du réseau (longueur NW), correspondant à la bande passante occupée par la session d'application active .
- iii) le temps moyen d'arrivée.

Les dispositifs de brouillard modèlent des nœuds de brouillard, c'est-à-dire des dispositifs capables d'héberger des modules d'application. La gestion des ressources est une composante essentielle d'IFogSim et est composée de Placement and Scheduler. Cette structure modulaire permet le déploiement de différents scénarios.

4.4.2 Simulateur FogTorch

Brogi et al. [38] proposent un modèle de Fog Computing qui se compose de profils QoS (qualité de service), d'infrastructures de Fog, d'applications IoT et de déploiements éligibles. Ce modèle est pris en charge par un outil de simulation nommé **FogTorch**. Le simulateur est écrit en Java et permet aux développeurs de personnaliser l'infrastructure de brouillard (en termes de cœurs de processeur, de RAM et de stockage par nœud, de définir des politiques de qualité de service liées à la latence et à la bande passante, et de spécifier les exigences de l'application.

4.4.3 Simulateur FogNetSim++

FogNetSim++ [39] est un cadre (Framework) de simulation capable de gérer les réseaux et les périphériques de Fog. Il est basé sur **OMNeT++** [40], une bibliothèque et un cadre de simulation C++ à base de composants open source, largement utilisé dans le milieu universitaire. **FogNetSim++** est un simulateur

axé sur les événements et son objectif principal est de fournir un environnement statique ou dynamique qui prend en charge les capteurs, les nœuds de brouillard, les centres de données distribués et un nœud de courtier. Le rôle de ce dernier est de gérer les autres appareils et leurs demandes. **FogNetSim++** prend en charge l'exécution d'algorithmes de planification des ressources tout en fournissant un modèle énergétique et différents modèles de tarification. Le modèle de tarification s'applique aux composants de réseau, de stockage et de calcul pour les tâches de réseau, de stockage, de calcul et autres. Le simulateur fournit aux utilisateurs des options de configuration détaillées. Il fournit également des modules pour la création de l'environnement réseau, et les utilisateurs peuvent intégrer leurs propres extensions. Les auteurs affirment que FogNetSim++ est évolutif en termes de temps d'exécution moyen, d'utilisation du processeur et de la mémoire.

4.4.4 Comparaison entre les différents simulateurs de Fog

Le tableau 4.2 exprime la comparaison des outils de simulation pour le Fog Computing en fonction de leurs capacités.

Simulateur	Language de programmation	Métrique	Ce qu'il fait	Ce qu'il ne fait pas	Infrastructure
iFogSim	Extension de Cloud-Sim, Java, JSON	Consommation d'énergie, congestion du réseau et coûts opérationnels	Comparaison des techniques de gestion des ressources, des topologies d'importation, calcul des coûts	Mobilité	Capteurs, actionneurs, dispositifs de Fog et centres de données
FogTorch	Java	QoS, fiabilité des liens et des nœuds, consommation d'énergie, sécurité, coûts monétaires	Déploiement d'applications IoT sensible à la QoS	La plupart des fonctionnalités manquent	Cloud, nœuds de Fog et autres choses
FogNetSim+	Basé sur OMNeT++	Module énergétique, algorithmes de planification, modèle de tarification	Fournit des modules intégrés (capteurs, appareils mobiles, nœuds de Fog, courtier), prend en charge les protocoles de communication, les applications	Migration de VM (machine virtuelle), autres types de coûts, propriétés du réseau réel (perte de paquets, congestion ou collision de canaux)	Appareils mobiles, nœuds de Fog, nœuds de courtier, capteurs et stations de base

TABLE 4.2 – Tableau comparatif entre des différents simulateurs de Fog.

4.5 CONCLUSION

Dans ce chapitre, nous avons présenté dans un premier temps des travaux existants sur le modèle d'architecture de l'information dans le Fog. Dans un second temps, nous détaillerons les algorithmes de positionnement des données sélectionnées. Ensuite, l'outil potentiel pour l'évaluation des modèles d'architecture dans le contexte du Fog Computing et de l'IoT. Après avoir présenté le contexte et la documentation en lien avec notre étude, nous présentons dans la prochaine section toutes nos contributions.

MODÈLE D'ARCHITECTURE PROPOSÉ ET SIMULATIONS

5

SOMMAIRE

5.1	INTRODUCTION	38
5.2	PROBLEMATIQUES	38
5.3	MODELE D'ARCHITECTURE PROPOSEE	38
5.3.1	Architecture Cloud-Fog	39
5.3.2	Caméras	39
5.3.3	Les nœuds de Fog	39
5.3.4	La couche Cloud	40
5.4	SIMULATION	40
5.4.1	Outils utilisés	40
5.4.2	Présentation du simulateur IFogSim	44
5.4.3	Classes utilisées	44
5.4.4	Stratégies de placement	46
5.4.5	Topologie	46
5.4.6	Implementation avant les résultats	48
5.4.7	Evaluation des résultats	51
5.4.8	Conclusion	57

5.1 INTRODUCTION

Le système de stationnement intelligent est une technologie qui vise à optimiser l'utilisation des espaces de stationnement en permettant aux conducteurs de trouver rapidement une place libre. Cette technologie utilise des capteurs pour détecter la présence de véhicules et des algorithmes pour guider les conducteurs vers des places disponibles. Cependant, pour que cette technologie fonctionne efficacement, elle nécessite un traitement en temps réel des données captées par les capteurs. C'est là que le Fog Computing intervient.

Le Fog Computing est une architecture informatique distribuée qui permet de traiter les données localement, au niveau des appareils connectés au réseau, plutôt que de les envoyer vers le Cloud. En utilisant le Fog Computing dans le système de stationnement intelligent, les données peuvent être traitées rapidement et efficacement, sans avoir besoin d'être transférées à des centres de données éloignés. Cette approche réduit les temps de latence et améliore la réactivité du système.

Dans cet travail, nous allons explorer en détail comment le Fog Computing peut être utilisé dans un système de stationnement intelligent pour améliorer ses performances et sa fiabilité. Nous examinerons les avantages et les défis du Fog Computing, ainsi que les différentes stratégies de déploiement des modules d'application.

5.2 PROBLEMATIQUES

L'objectif principal de notre recherche étant, de trouver la meilleure solution possible pour les problèmes cités ci-dessus. Ceci, en suggérant un environnement intelligent basé sur l'architecture Cloud-Fog pouvant assurer les améliorations suivantes :

- Un traitement efficace pour les applications de stationnement intelligent basées sur l'IoT.
- Une utilisation maximale des ressources et en un temps de réponse très réduit.

5.3 MODELE D'ARCHITECTURE PROPOSEE

L'architecture proposée se compose de trois couches comme illustré dans la Figure 5.1.

La première couche contient les caméras et est responsable de la capture des images des emplacements de stationnement et déterminer si l' emplacement de stationnement est occupé ou non.

La deuxième couche du système est un nœud de brouillard(nœud de Fog) qui est connecté aux caméras via un dispositif à microcontrôleur.

La troisième couche contient un nuage(Cloud) qui est connecté au brouillard(nœud de Fog) et est responsable du stockage et de la gestion des données des images pendant des périodes plus longues.

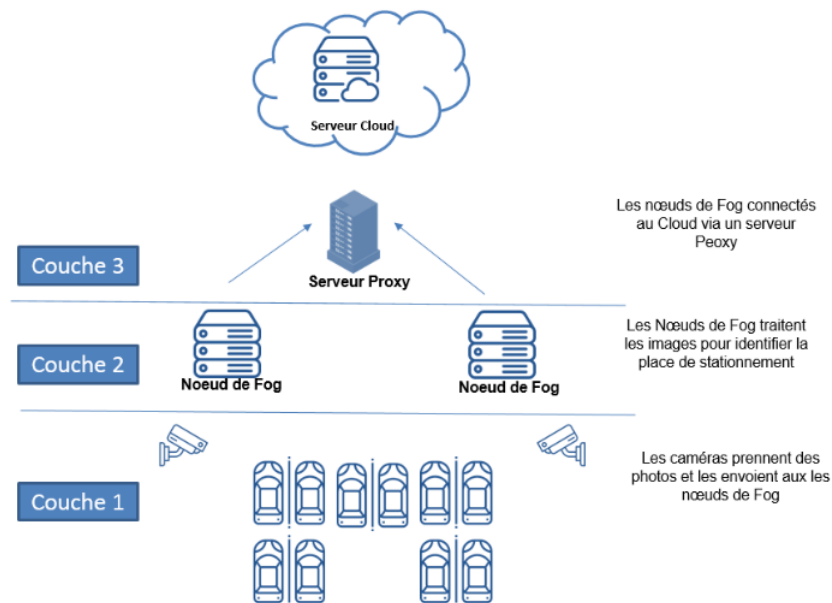


FIGURE 5.1 – Architecture à trois niveaux pour le système de stationnement intelligent

5.3.1 Architecture Cloud-Fog

La solution proposée consiste en un système basé sur l'architecture Cloud-Fog a 3 niveaux :

5.3.2 Caméras

Les caméras constituent le premier niveau du système smart parking. Nous avons préféré les caméras aux capteurs pour détecter les places de stationnement vacantes pour la raison que les capteurs dépendent fortement de la position du véhicule stationné pendant que l'approche basée sur le traitement d'image identifie et localise l'emplacement de stationnement vacant sur la base de segments de lignes de stationnement.

5.3.3 Les nœuds de Fog

C'est une couche intermédiaire entre la couches des caméras et le Cloud, elle est utilisée pour collecter les données d'image de la caméra. Le microcontrôleur détecte l'état des emplacements de stationnement équipés de parcmètres. Une place de parking a un index spécifique, par exemple, **PS₁₁** signifie rangée 1 et emplacement 1 dans une voie de stationnement particulière. Le brouillard met à jour l'état de la place de stationnement à la lampe (LED) intelligente, située à l'entrée du Parking.

5.3.4 La couche Cloud

Dans cette architecture, le Cloud est responsable du stockage de grandes quantités de données, du traitement des flux de sortie pour l'analyse des données collectées sur une longue période. La communication entre le Fog et le Cloud est activée via un serveur Proxy. Le nœud de Fog passe les données d'images vers le Cloud après une période de temps spécifique, et si le Fog a besoin de données des images, les données seront fournies par le Cloud.

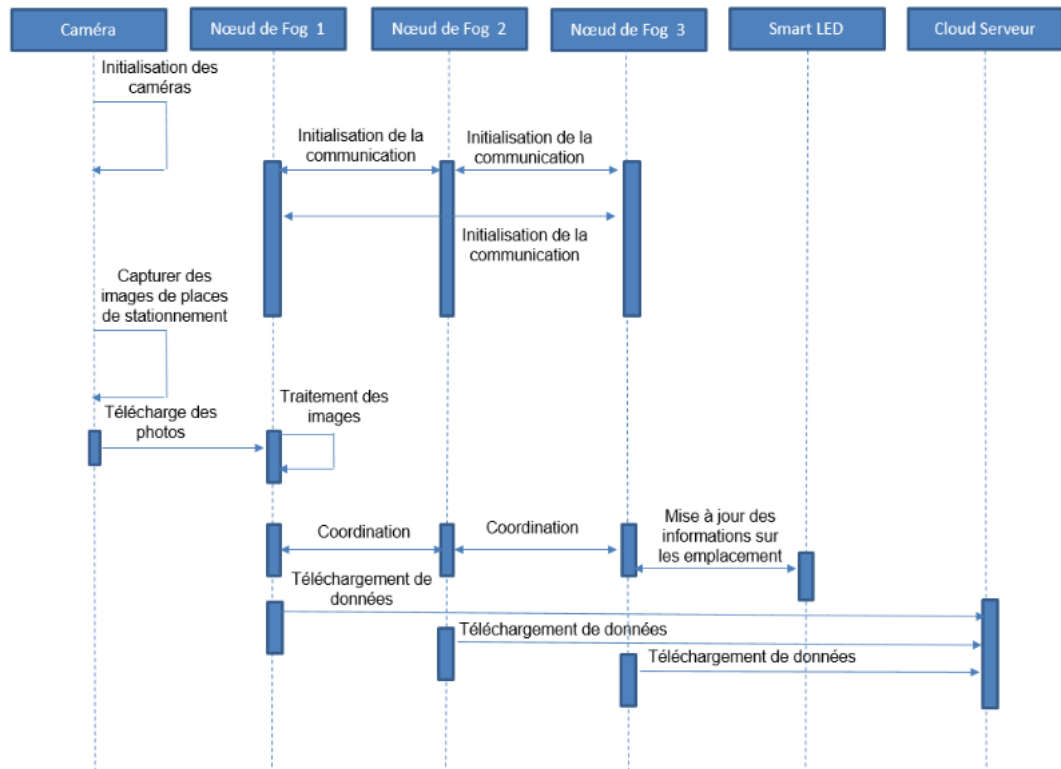


FIGURE 5.2 – Interaction entre les caméras, les Fog nœuds et le Cloud Data Center

5.4 SIMULATION

5.4.1 Outils utilisés

L'un des grands problèmes du développement d'applications sur le Cloud est la reproduction des tests dans un environnement approprié. Effectuer des tests sur un environnement Cloud réel tel qu'Amazon S3 peut s'avérer très coûteux. Cependant, grâce à la simulation, les développeurs peuvent tester et optimiser leurs applications Cloud sans avoir à payer. Même les clients peuvent tester les performances de leurs services de manière reproductible, dans un environnement contrôlable et gratuit avant de déployer les applications sur l'environnement Cloud réel. Un simulateur Cloud est un outil qui permet de manipuler des Data-Center avec leurs serveurs de base et le matériel nécessaire à leur bon

fonctionnement (CPU, disque dur, RAM, etc.), afin de simuler la création et la distribution des tâches sur des machines virtuelles.

a. CloudSim

CloudSim est une plateforme de simulation pour l'environnement Cloud qui permet de modéliser des DataCenter avec leurs serveurs de base et les machines virtuelles. Pour utiliser CloudSim, vous devez suivre les étapes suivantes :

- 1- Télécharger CloudSim depuis le site officiel.
- 2- Installer Java Development Kit (JDK) si vous ne l'avez pas déjà installé sur votre ordinateur.
- 3- Importer les bibliothèques CloudSim dans votre projet Java.
- 4- Écrire le code Java pour définir votre simulation Cloud. Cela implique de créer des DataCenter, de définir les serveurs et les machines virtuelles, d'instancier des stratégies de placement de machines virtuelles et de planification des tâches.
- 5- Exécuter votre simulation Cloud en utilisant la méthode "startSimulation" de la classe CloudSim.
- 6- Analyser les résultats de votre simulation en utilisant les outils de visualisation fournis par CloudSim.

Il est important à noter que l'utilisation de CloudSim nécessite des connaissances en programmation Java et en environnement Cloud.

a.1 Caractéristiques du CloudSim

Les principales caractéristiques de CloudSim sont les suivantes :

- 1- Il fournit une plate-forme de simulation de haut niveau pour modéliser et simuler divers types d'environnements de Cloud Computing.
- 2- Il prend en charge la modélisation des ressources de Cloud Computing, telles que les serveurs, les machines virtuelles, les centres de données, etc.
- 3- Il permet la simulation de différents types d'applications de Cloud Computing, tels que les applications Web, les applications mobiles, etc.
- 4- Il prend en charge la modélisation de diverses politiques de gestion de ressources de Cloud Computing, telles que les politiques de placement de machine virtuelle, les politiques d'équilibrage de charge, etc.
- 5- Il fournit une bibliothèque de composants pré-construits pour faciliter la modélisation et la simulation des environnements de Cloud Computing.
- 6- Il prend en charge la simulation de scénarios complexes de Cloud Computing, tels que la migration de machines virtuelles, l'ajout et la suppression dynamiques de ressources, etc.
- 7- Il permet l'analyse des performances de l'environnement de Cloud Computing simulé en collectant des données statistiques sur l'utilisation des ressources, le délai de traitement, la consommation d'énergie, etc.
- 8- Il est écrit en Java, ce qui facilite l'intégration avec d'autres outils et bibliothèques Java.

a.2 Limites du CloudSim

CloudSim est un outil de simulation largement utilisé pour évaluer les performances des infrastructures de Cloud Computing. Cependant, il présente certaines limites qui peuvent affecter la précision des résultats de simulation.

Les modèles de consommation d'énergie, de réseau et de machines virtuelles utilisés par CloudSim sont simplistes et ne prennent pas en compte les variations de la charge de travail ou les différences de consommation d'énergie et de performances entre les différents types de serveurs et de machines virtuelles.

De plus, CloudSim peut être lent pour simuler de grands systèmes avec un grand nombre de machines virtuelles et d'utilisateurs, ce qui peut affecter l'évolutivité de l'outil.

La configuration de CloudSim peut également être complexe pour les utilisateurs débutants, car elle nécessite une connaissance approfondie des paramètres de simulation tels que le temps de simulation, la topologie du réseau et les modèles de machines virtuelles.

Enfin, la précision des résultats de simulation dépend de la qualité des modèles de simulation et de la précision des entrées de simulation. Bien que CloudSim reste un outil de simulation précieux pour évaluer les performances des infrastructures de Cloud Computing, il est important de prendre en compte ces limites lors de l'interprétation des résultats de simulation.

b. IFogSim

IFogSim est un outil de simulation OpenSource conçu pour simuler les infrastructures de Fog Computing. Il utilise une approche basée sur des agents pour simuler les interactions entre les différents appareils et services du réseau de Fog Computing. IFogSim prend en compte des facteurs tels que la latence, la bande passante, la disponibilité des ressources et les coûts énergétiques pour évaluer les performances du réseau. Il présente des avantages par rapport à CloudSim, notamment sa capacité à simuler des infrastructures de Fog Computing plus complexes et sa prise en compte de facteurs tels que la disponibilité des ressources et les coûts énergétiques. Cependant, IFogSim peut être complexe à configurer et nécessite une connaissance approfondie de l'architecture de Fog Computing pour interpréter les résultats de simulation. Pour cet effet nous avons opté cet outil comme un simulateur de référence. La figure 5.3 présente l'interface d'accueil de IFogSim.

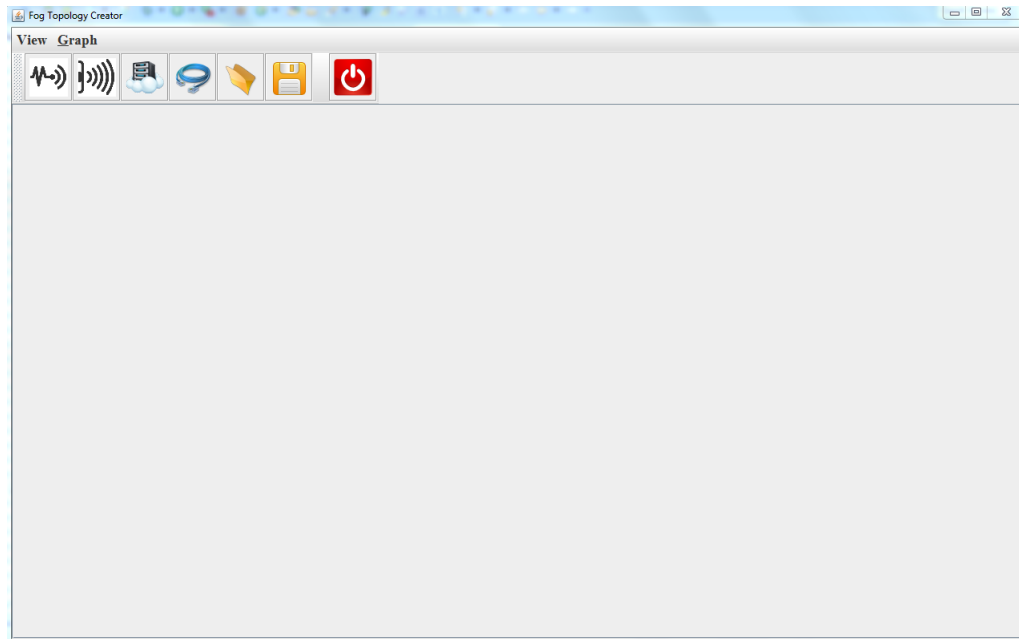


FIGURE 5.3 – Interface de IFogSim

Langage de programmation utilisé

Dans le cadre de notre projet nous avons utilisé le langage de programmation java.

Eclipse Java

Eclipse est un environnement de développement intégré (IDE) OpenSource pour plusieurs langages de programmation, y compris Java. Il est utilisé pour le développement de logiciels professionnels en Java, offrant des fonctionnalités telles que l'édition de code, la compilation, le débogage et la gestion de projets. L'IDE Eclipse Java comprend des fonctionnalités telles que la complétion de code, le refactoring, la gestion des bibliothèques, la détection d'erreurs et la génération automatique de code, ce qui facilite le développement d'applications Java complexes. Eclipse prend également en charge de nombreux plugins qui peuvent être ajoutés pour améliorer ses fonctionnalités, tels que des Plugins pour les outils de test unitaire, les outils de développement Web, les outils de contrôle de version, etc. Eclipse est un IDE populaire pour Java et est largement utilisé par les développeurs Java dans le monde entier en raison de ses fonctionnalités riches et de sa grande flexibilité.

- Afin de réaliser cette simulation, nous avons utilisé un ordinateur équipé de :

Processeur : Intel(R) Core(TM) i3-4005U CPU @ 1.70GHz 1.70 GHz

Mémoire (RAM) : 4,00 Go

Type du système : Système d'exploitation 64 bits ,processeur x64

Édition Windows : Windows 8.1 Professionnel

5.4.2 Présentation du simulateur IFogSim

L'utilisation du Fog Computing dans le système de stationnement intelligent présente plusieurs avantages pour les utilisateurs et les opérateurs du système.

- Les capteurs de stationnement et les caméras sont installés sur les emplacements de stationnement pour collecter des données sur les lieux disponibles en temps réel.
- Les données nécessaires sont hébergées dans les nœuds de calcul situés à proximité des capteurs et des caméras, plutôt que d'être transmises à un centre de données distant pour le traitement.
- Les nœuds de calcul peuvent effectuer des analyses préliminaires sur les données fournies, telles que la détection des véhicules stationnés et la vérification de la disponibilité des places de stationnement.
- Les données sont ensuite envoyées aux serveurs Fog pour un traitement supplémentaire et une analyse plus approfondie.
- Les serveurs Fog peuvent fournir des informations aux utilisateurs, telles que la localisation des places de stationnement disponibles, les horaires de pointe pour la demande de stationnement et les zones de stationnement les plus populaires.
- L'utilisation du Fog Computing réduit la latence et améliore la vitesse de traitement des données, ce qui permet aux utilisateurs de trouver rapidement des places de stationnement disponibles.
- De plus, l'utilisation du Fog Computing a réduit les coûts de transmission de données en traitant les données localement, ce qui améliore l'efficacité globale du système.
- Enfin, l'application contribue à réduire la congestion et la pollution dans les villes en aidant les utilisateurs à découvrir rapidement les places de stationnement disponibles, ce qui réduit la circulation routière et les émissions de gaz d'échappement.

5.4.3 Classes utilisées

Les différentes entités qui constituent le simulateur sont mises en œuvre à l'aide des classes suivantes :

AppModule

Les éléments de traitement des applications Fog sont représentés par des instances de la classe AppModule. À chaque fois qu'un tuple entre, une instance de AppModule le traite et produit des tuples de sortie qui sont ensuite transmis aux modules suivants du graphe. En somme, les modules AppModule sont essentiels au traitement efficace des applications Fog.

AppEdge

La classe AppEdge représente la dépendance des données entre une paire de modules d'application (source et destination). Chaque AppEdge est caractérisée par le type de tuple qu'il transporte, qui est capturé par l'attribut "Type" de la

classe "AppEdge", ainsi que par les exigences de traitement (CPU) et la longueur des données encapsulées dans ces tuples. En somme, les instances d'AppEdge sont des éléments clés qui permettent d'assurer une transmission efficace des données entre les différents modules d'une application.

AppLoop

La classe AppLoop est utilisée pour spécifier les boucles de contrôle de processus qui sont pertinentes pour l'utilisateur. Dans le cadre d'iFogSim, le développeur peut spécifier les boucles de contrôle afin de mesurer la latence End-to-End. En substance, une instance de la classe AppLoop est une liste de modules allant de l'origine de la boucle au module où celle-ci se termine. Cette classe est donc essentielle pour permettre aux développeurs de concevoir des boucles de contrôle efficaces pour leurs applications.

FogDevice

La classe FogDevice permet de spécifier les caractéristiques matérielles des dispositifs Fog et leur connexion avec d'autres dispositifs, capteurs et actionneurs. Les attributs principaux de cette classe incluent la mémoire, le processeur, la capacité de stockage, ainsi que les largeurs de bande montante et descendante, qui déterminent la capacité de communication des périphériques Fog. Les méthodes de cette classe permettent de planifier efficacement les ressources d'un dispositif Fog entre les différents modules d'application qui y sont exécutés et de déployer les modules de manière optimale. En somme, la classe FogDevice est un élément clé dans la conception et l'optimisation des infrastructures Fog.

Capteur

Les instances de la classe Sensor représentent des capteurs IdO qui agissent comme des entités de la plateforme Fog. La classe possède un attribut qui fait référence au dispositif Fog de la passerelle auquel le capteur est connecté, ainsi que la latence de connexion entre eux. La classe Sensor définit également les caractéristiques de sortie du capteur et la distribution temporelle entre les transmissions ou les arrivées des tuples, qui identifie le débit d'arrivée des tuples au niveau de la passerelle.

Actionneur

La classe Actionneur représente un composant du système qui peut effectuer des actions en réponse à l'arrivée de tuples. Elle modélise les propriétés de connexion réseau de l'actionneur, en contenant un attribut de référence à la passerelle à laquelle il est connecté et la latence de cette connexion. De plus, elle définit une méthode qui spécifie l'action à effectuer à l'arrivée d'un tuple depuis un module d'application.

Tuple

Les échanges de données au sein du Fog sont effectués au moyen de tuples, qui représentent l'unité de base de communication entre les entités. Chaque tuple est caractérisé par son type, ainsi que par les modules d'application source et cible. Les exigences de traitement liées à chaque tuple sont spécifiées par les attributs de la classe correspondante.

5.4.4 Stratégies de placement

Nous avons déjà étudié les stratégies de placement des modules d'application : la stratégie de placement exclusivement dans le Cloud (isOnlyCloud) et la stratégie de placement dans le Fog (isFog). La stratégie de placement Cloud implique la mise en œuvre traditionnelle des applications dans le Cloud, où tous les modules de l'application sont exécutés dans des centres de données. Dans cette stratégie, les capteurs transmettent les données détectées au Cloud pour être traitées, et les actionneurs sont informés si une action est une exigence.

D'un autre côté, la stratégie de placement Fog favorise le déploiement de modules d'application à proximité du réseau. Cependant, les périphériques proches du réseau, tels que les routeurs et les points d'accès, peuvent ne pas avoir suffisamment de puissance de calcul pour éberger tous les opérateurs de l'application. Dans ce cas, la stratégie parcourt les périphériques Fog vers le Cloud et tente de placer les opérateurs restants sur des périphériques alternatifs.

5.4.5 Topologie

Le tableau 5.1 montre la configuration des périphériques utilisés dans le réseau.

Paramètres	Cloud	Proxy-Server	Fog
Longueur du processeur (CPU)	44800	2800	2800
RAM (MB)	40000	4000	4000
Bande passante de liaison montante (MB)	100	10000	10000
Bande passante descendante (MB)	10000	10000	10000
Niveau	0	1	2
Taux Par Mips	0.01	0.0	0.0
Puissance occupée (Watt)	16*103	107.339	107.339
Puissance inactive	16*83.25	83.43	83.43

TABLE 5.1 – Configuration des appareils dans la topologie.

- La figure 5.4 représente la topologie créée dans iFogSim pour évaluer les résultats du scénario de Fog. Nous avons créé trois nœuds de Fog dans cette topologie et trois caméras ont été attachées à chaque nœud de Fog. Cette topologie a été créée pour évaluer la latence et l'utilisation du réseau dans iFogSim. Nous avons intégré le module "picture-capture" dans les caméras pour capturer les images des emplacements de parking. Nous avons créé le module de détection des emplacements et l'avons intégré dans les nœuds de Fog pour traiter les images afin d'obtenir des informations sur les emplacements. Pour traiter les images afin de détecter les places de parking vacantes. En outre, le nœud de Fog met à jour l'état des emplacements de parking à la LED intelligente connectée.

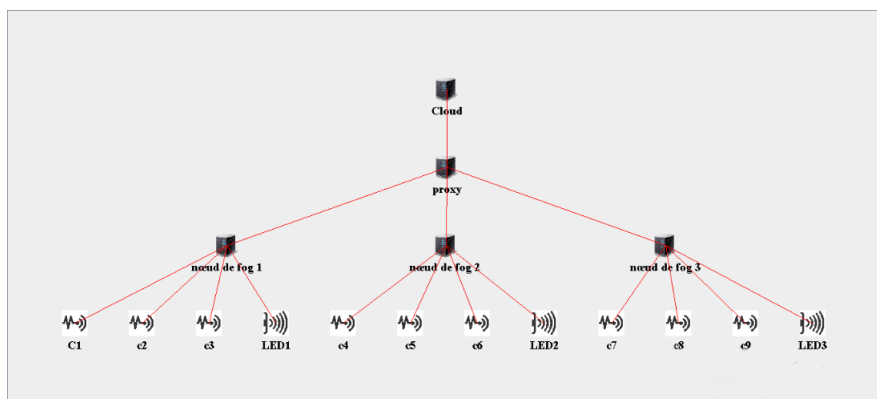


FIGURE 5.4 – Topologie iFogSim de 9 caméras connectées à 3 nœuds de Fog connectés au serveur Cloud via le serveur proxy.

- La figure 5.5 illustre la configuration en Cloud d'iFogsim. Pour l'évaluation des performances dans le scénario basé sur le Cloud, plusieurs caméras et LED sont connectées au serveur Cloud par l'intermédiaire d'un routeur. Les caméras envoient les images des emplacements de parking au serveur Cloud. Le serveur Cloud traite les images des emplacements de parking et les informations relatives aux emplacements de parking esr seront affichées sur la LED qui est connectée au serveur Cloud. Le nombre de Le nombre de caméras a été progressivement augmenté afin d'examiner la valeur de la latence et de l'utilisation du réseau.

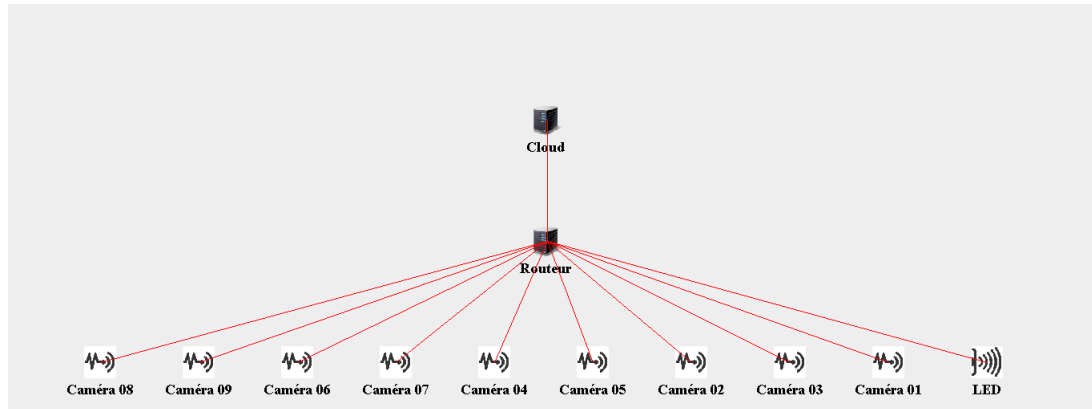


FIGURE 5.5 – Topologie iFogSim d'un scénario basé sur isOnlyCloud avec des caméras connectées par l'intermédiaire d'un routeur.

5.4.6 Implementation avant les résultats

La figure 5.6 montre le code nécessaire pour l'instanciation et le stockage des appareils de brouillard (Fog Devices), des capteurs et des actionneurs sont effectués sous forme de listes de tableaux (Array listes) et la création des modules (machines virtuelles) dans les nœuds de Fog (Fog Nodes) (caméras, routeurs) est réalisée à l'aide du module de mappage (Mapping) et du module de mappage de bord (Edge Mapping). Si un appareil de Fog (Fog Device) a une relation avec une zone, on peut y ajouter un module appelé "Slot-Detector". Si l'appareil de Fog est une caméra, on peut y ajouter un module appelé "Picture-Capture".

```

package org.fog.test.perfeval;
import java.util.ArrayList;
public class smartCarParkingFog {
    static List<FogDevice> fogDevices = new ArrayList<FogDevice>();
    static List<Sensor> sensors = new ArrayList<Sensor>();
    static List<Actuator> actuators = new ArrayList<Actuator>();
    static int numOfAreas = 1;
    static int numOfCamerasPerArea = 25;
    static double CAM_TRANSMISSION_TIME = 5;
    private static boolean CLOUD = false;
    public static void main(String[] args) {
        Log.println("Starting smart car parking system...");
        try {
            Log.disable();
            int num_user = 1;
            Calendar calendar = Calendar.getInstance();
            boolean trace_flag = false;
            CloudSim.inir(num_user, calendar, trace_flag);
            String appId = "samrt parking";
            FogBroker broker = new FogBroker("broker");
            Application application = createApplication(appId, broker.getId());
            application.setUserId(broker.getId());
            createFogDevices(broker.getId(), appId);
            Controller controller = null;
            ModuleMapping moduleMapping = ModuleMapping.createModuleMapping();
            for(FogDevice device : fogDevices){
                if(device.getName().startsWith("c")){
                    moduleMapping.addModuleToDevice("picture-capture", device.getName());
                }
            }
            for(FogDevice device : fogDevices){
                if(device.getName().startsWith("a")){
                    moduleMapping.addModuleToDevice("slot-detector", device.getName());
                }
            }
        }
    }
}

```

FIGURE 5.6 – Code principal .

Le code source dans la Figure 5.7 est nécessaire pour la création des appareils de brouillard (Fog Devices) tels que le Cloud et le Proxy sont réalisées en spécifiant le niveau à l'aide de la méthode "createFogDevices". Cette méthode permet de créer un nœud de brouillard (Fog Node) pour chaque zone (Area).

```

private static void createFogDevices(int userId, String appId) {
    FogDevice cloud = createFogDevice("cloud", 44800, 40000, 100, 10000, 0, 0.01, 16*103, 16*83.25);
    cloud.setParentId(-1);
    fogDevices.add(cloud);
    FogDevice proxy = createFogDevice("proxy-server", 2800, 4000, 10000, 10000, 1, 0.0, 107.339, 83.4333);
    proxy.setParentId(cloud.getId());
    proxy.setUplinkLatency(100);
    fogDevices.add(proxy);
    for(int i=0; i<numOfAreas; i++){
        addArea(i+"", userId, appId, proxy.getId());
    }
}

```

FIGURE 5.7 – Création des appareils de brouillard (Fog Devices).

Les figures 5.8 et 5.9 exposent le code qui détaille la création des caméras, l'intégration des capteurs et des actionneurs, ainsi que des routeurs, tous étant des appareils de brouillard (Fog Devices).

```

private static FogDevice addArea(String id, int userId, String appId, int parentId){
    FogDevice router = createFogDevice("a-"+id, 2800, 4000, 1000, 10000, 1, 0.0, 107.339, 83.4333);
    fogDevices.add(router);

    for(int i=0;i<numOfCamerasPerArea;i++){
        String mobileId = id+"-"+i;
        FogDevice camera = addCamera(mobileId, userId, appId, router.getId());
        camera.setUpLinkLatency(2);
        fogDevices.add(camera);
    }
    router.setParentId(parentId);
    return router;
}

```

FIGURE 5.8 – Création des routeurs.

```

private static FogDevice addCamera(String id, int userId, String appId, int parentId){
    FogDevice camera = createFogDevice("c-"+id, 500, 1000, 10000, 10000, 2, 0, 87.53, 82.44);
    camera.setParentId(parentId);
    Sensor sensor = new Sensor("s-"+id, "CAMERA", userId, appId, new DeterministicDistribution(CAM_TRANSMISSION_T));
    sensors.add(sensor);
    Actuator ptz = new Actuator("ptz-"+id, userId, appId, "PTZ_CONTROL");
    actuators.add(ptz);
    sensor.setGatewayDeviceId(camera.getId());
    sensor.setLatency(40.0);
    ptz.setGatewayDeviceId(parentId);
    ptz.setLatency(1.0);
    return camera;
}

```

FIGURE 5.9 – Création des caméras.

Pour créer des applications, il est nécessaire d'ajouter un module d'application en utilisant la fonction "addAppModule", comme illustré dans la figure 5.10. Cette fonction permet d'établir la relation entre les Fog Devices et les modules en spécifiant la source, la destination, la sortie et le NWTuple (Network Tuple) dans la fonction "addAppEdge". De plus, la fonction "addTupleMapping" est utilisée pour définir la relation d'entrée-sortie d'un module d'application pour un type de tuple d'entrée spécifique. Pour déterminer la latence et la bande passante du réseau pour un utilisateur, la fonction "AppLoop" est utilisée.

```

@SuppressWarnings({"serial" })
private static Application createApplication(String appId, int userId){

    Application application = Application.createApplication(appId, userId);

    application.addAppModule("picture-capture", 10);
    application.addAppModule("slot-detector", 10);

    application.addAppEdge("CAMERA", "picture-capture", 1000, 500, "CAMERA", Tuple.UP, AppEdge.SENSOR);
    application.addAppEdge("picture-capture", "slot-detector",
        1000, 500, "slots", Tuple.UP, AppEdge.MODULE);

    application.addAppEdge("slot-detector", "PTZ_CONTROL", 100,
        28, 100, "PTZ_PARAMS",
        Tuple.UP, AppEdge.ACTUATOR);
    application.addTupleMapping("picture-capture", "CAMERA", "slots",
        new FractionalSelectivity(1.0));
    application.addTupleMapping("slot-detector", "slots",
        "PTZ_PARAMS", new FractionalSelectivity(1.0));
    final AppLoop loop1 = new AppLoop(new ArrayList<String>()
    {{add("CAMERA");
    add("picture-capture");add("slot-detector");
    add("PTZ_CONTROL");}}});

    List<AppLoop> loops = new ArrayList<AppLoop>() {{add(loop1)}};
    application.setLoops(loops);
    return application;
}
    
```

FIGURE 5.10 – Création des applications .

5.4.7 Evaluation des résultats

Les résultats obtenus concernant la latence, l'utilisation du réseau, le temps d'exécution et la consommation d'énergie pour la simulation des deux modèles d'architecture , à savoir "isOnlyCloud" et "isOnlyFog", sont interprétés de la manière suivante.

a- Latence :

où α est le délai d'exécution du Tuple CPU pour la capture des images et μ est le temps nécessaire pour télécharger les images sur le nœud de Fog en vue de leur traitement et de leur stockage. Enfin, ϕ est le temps nécessaire pour afficher les informations sur la LED après traitement au nœud de Fog.

$$latence = \alpha + \mu + \phi \quad (5.1)$$

Le tableau 5.2 présente les résultats de la latence dans les environnements de brouillard (Fog) et de Cloud, en fonction du nombre de caméras.

Caméras	Latence du Fog (ms)	Latence du Cloud (ms)
5	50.1327	254.0706
10	51.9185	246.316
15	1706.73334	1871.4831
20	2535.8169	2684.0665
25	3033.2671	3171.6165
30	3498.3360	3469.6498
35	3730.4552	3728.8160
40	3904.5438	3902.9404

TABLE 5.2 – Les résultats de la simulation, visant à évaluer la latence de l'architecture de parking proposée dans les environnements Fog et Cloud.

- La comparaison de la latence entre le Cloud et l'environnement de Fog est présentée dans la Figure 5.11. Les résultats montrent clairement qu'au fur et à mesure que le nombre de caméras augmente, la latence dans le Cloud augmente de manière significative par rapport au Fog. Cette différence s'explique par le fait que dans le Fog, le nœud de Fog dédié à une zone particulière traite uniquement les images de cette zone. En revanche, dans le Cloud, le serveur Cloud traite les images de toutes les zones de stationnement, ce qui entraîne une augmentation de la latence lorsque le nombre de caméras augmente.

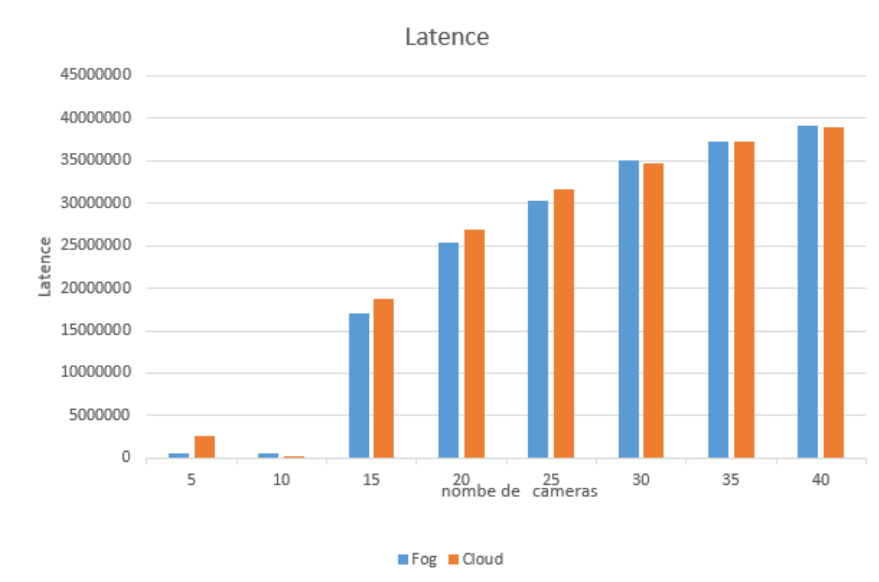


FIGURE 5.11 – Impact de la variation du nombre de caméras sur la latence dans les deux modèles proposés (isOnlyCloud) et (isOnlyFog).

b- Utilisation du réseau :

$$Networkusage = latence * \partial \quad (5.2)$$

où

$$\partial = tupleNWSize \quad (5.3)$$

Le tableau 5.3 présente les résultats de l'utilisation du réseau dans l'environnement de Fog par rapport au nombre de caméras.

Caméras	Utilisation du réseau de Fog (Kb)	Utilisation du réseau de Cloud (Kb)
5	995	60723
10	1990	121319
15	2985	122315
20	3980	123311
25	3 4975	124307
30	128115	125303
35	129600	126299
40	131085	127295

TABLE 5.3 – Les résultats de la simulation, visant à évaluer l'utilisation du réseau de l'architecture de parking proposée dans les environnements Fog et Cloud.

- La Figure 5.12 illustre les résultats de l'utilisation du réseau pour le scénario de Fog. Il est clairement visible dans la Figure 5.12 que l'augmentation de caméras entraîne une augmentation de l'utilisation du réseau. L'augmentation de l'utilisation du réseau dans l'environnement de Cloud s'explique par le fait que lorsque toutes les caméras sont connectées au serveur Cloud, toutes les tuples sont traitées par un seul serveur Cloud à la fois, ce qui entraîne une augmentation de l'utilisation du réseau. En revanche, dans le cas du Fog, plusieurs caméras sont connectées à plusieurs nœuds de Fog, où chaque nœud de Fog est dédié à une zone de stationnement spécifique. Par conséquent, un nœud de Fog est uniquement responsable du traitement des tuples des caméras déployées dans cette zone particulière.

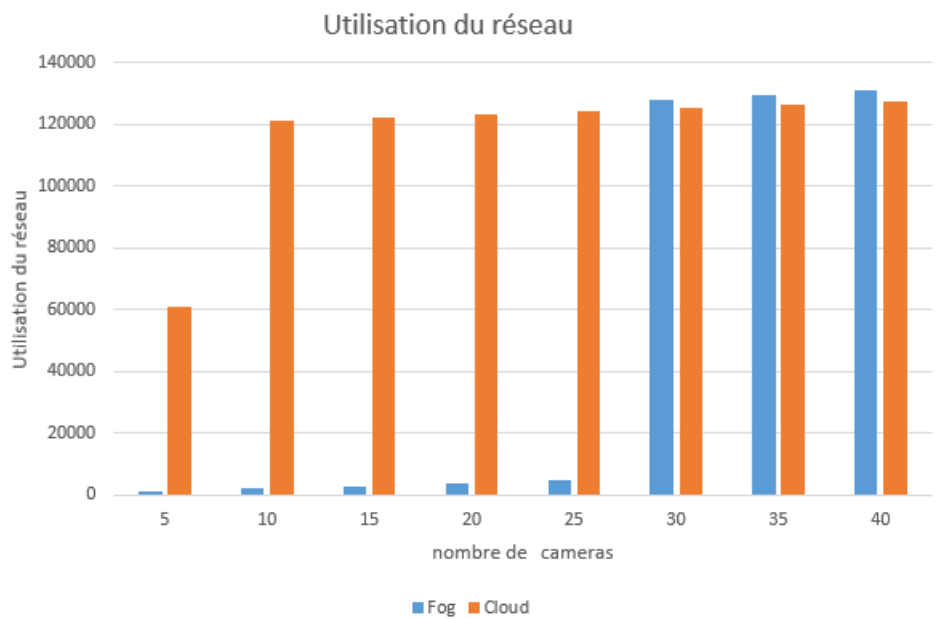


FIGURE 5.12 – Impact de la variation du nombre de cameras sur l'utilisation du réseau dans les deux modèle proposés (isOnlyCloud) et (isOnlyFog).

c- Le temps d'exécution :

Le temps d'exécution est la somme des temps d'exécution de tous les modules des applications exécutées. Il comprend le temps de traitement de l'algorithme de placement et la latence entre les dispositifs utilisés, depuis la sortie du capteur jusqu'au retour des actionneurs.

Les résultats de le temps d'exécution dans l'environnement de Fog en relation avec le nombre de caméras sont exposés dans le tableau 5.4.

caméras	Le temps d'exécution dans Fog (ms)	Le temps d'exécution dans Cloud (ms)
5	2972	3947
10	4417	7154
15	6270	7318
20	6681	9079
25	7358	11781
30	9317	13812
35	10823	14603
40	11194	17781

TABLE 5.4 – Les résultats de la simulation, visant à évaluer le temps d'exécution de l'architecture de parking proposée dans les environnements Fog et Cloud.

- Les résultats de le temps d'exécution pour le Fog et le Cloud sont présentés dans la Figure 5.13. On peut clairement observer dans cette figure que l'augmentation du nombre de caméras entraîne une hausse de le temps d'exécution. La croissance de le temps d'exécution dans le contexte du Cloud Computing peut être interprétée à la distance entre les utilisateurs et les données.

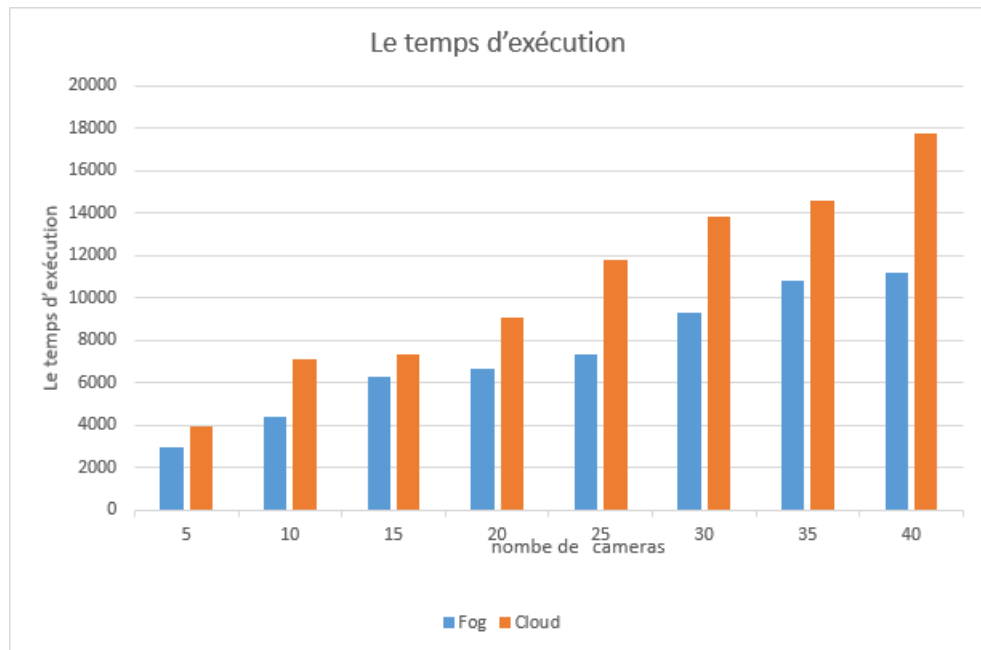


FIGURE 5.13 – Impact de la variation du nombre de caméras sur le temps d'exécution dans les deux modèle proposés (isOnlyCloud) et (isOnlyFog).

d- La consommation d'énergie :

La consommation d'énergie d'un dispositif de Fog est calculée en fonction de la puissance de tous les hôtes au cours d'une certaine période d'exécution. La méthode "UpdateEnergyConsumption" de la classe FogDevice est utilisée pour calculer la consommation d'énergie ; il s'agit d'une extension de la classe PowerDatacenter.

Les résultats de la consommation d'énergie dans l'environnement de brouillard (Fog) et dans le Cloud sont présentés dans le tableau 5.5, en fonction du nombre de caméras.

caméras	La consommation d'énergie dans Fog (Kb)	La consommation d'énergie dans Cloud (Kb)
5	1.34610	1.41051
10	1.34610	1.48604
15	1.34610	1.48604
20	1.34610	1.48604
25	1.34610	1.48604
30	1.34610	1.48604
35	1.34610	1.48604
40	1.34610	1.48604

TABLE 5.5 – Les résultats de la simulation, visant à évaluer la consommation d'énergie de l'architecture de parking proposée dans les environnements Fog et Cloud.

- Les résultats de la consommation d'énergie par différentes catégories d'appareils dans la simulation sont présentés dans la Figure 5.14. Une comparaison a été effectuée entre le Fog et le Cloud. Dans la Figure 5.14, on peut clairement observer que l'augmentation du nombre de caméras entraîne une hausse de la consommation d'énergie. Cette augmentation de la consommation d'énergie est constatée dans l'environnement de Cloud se justifie par le fait que le Cloud est doté de capacités de traitement élevées, en revanche, le Fog dispose d'une puissance de traitement des données relativement plus faible par rapport au Cloud et c'est ce qui explique la diminution de la consommation d'énergie.

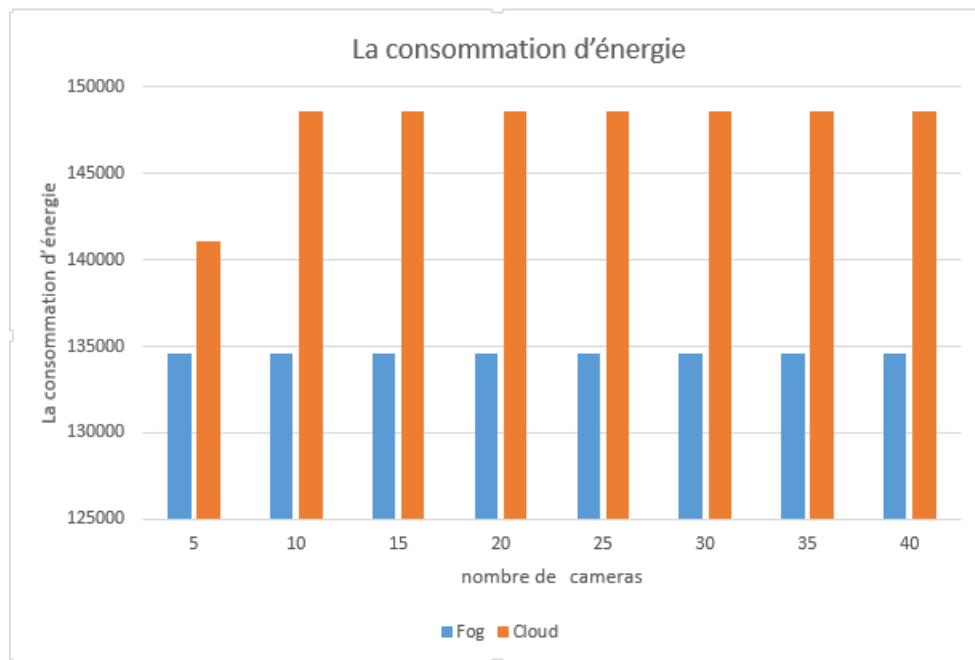


FIGURE 5.14 – Impact de la variation du nombre de caméras sur la consommation d'énergie dans les deux modèles proposés (isOnlyCloud) et (isOnlyFog).

5.4.8 Conclusion

Dans cette section, nous avons exposé les résultats des simulations effectuées en fixant certains paramètres et en faisant varier d'autres. Notre choix s'est porté sur l'étude d'un système de stationnement intelligent. Les résultats obtenus nous ont permis de valider la viabilité de l'architecture de déploiement proposée, qui se base sur l'utilisation des Fog.

Au cours des dernières années, le Fog Computing a joué un rôle crucial, notamment dans les domaines d'application nécessitant une réponse rapide. Avec la prolifération des appareils générant des données, le besoin d'une réactivité accrue s'est accentué. Dans cette perspective, nous avons présenté une architecture de parking intelligent basée sur le Fog Computing. Cette architecture utilise une technique de vision par ordinateur pour détecter les places de parking disponibles, permettant ainsi aux conducteurs de les trouver rapidement, ce qui réduit le temps et la consommation de carburant. Les résultats expérimentaux démontrent que notre architecture basée sur le Fog Computing minimise la latence et réduit l'utilisation du réseau, le temps d'exécution et la consommation d'énergie par rapport à une architecture basée sur le Cloud Computing. Cependant, certaines limitations ont été identifiées dans cette recherche. L'utilisation de caméras pour la détection des places de stationnement peut poser des problèmes de confidentialité pour les propriétaires de voitures. Des mesures doivent être prises pour garantir la protection des données stockées dans le Cloud, notamment par le biais de techniques de chiffrement adaptées. De plus, à mesure que le système de stationnement intelligent s'étend à un plus grand nombre de zones de stationnement, il sera essentiel de mettre en place un équilibrage de charge efficace pour maintenir les performances et l'efficacité du réseau de brouillard.

Dans l'ensemble, cette étude de cas démontre le potentiel du Fog Computing dans le domaine des systèmes de stationnement intelligent. Des efforts supplémentaires sont nécessaires pour résoudre les problèmes de confidentialité et d'équilibrage de charge, mais les avantages potentiels en termes de latence réduite et d'efficacité accrue en font une approche prometteuse pour l'avenir des solutions de stationnement intelligent.

BIBLIOGRAPHIE

- [1] V. Gazis, A. Leonardi, K. Mathioudakis, Konstantinos Sasloglou, Panayotis Kikiras, and Raghuram Sudhaakar. Components of fog computing in an industrial internet of things context. Fog networking for 5G and IoT Workshop 2015 (IEEE SECON Workshop) , pages 1–7.
- [2] A Thierer, The Internet of Things and Wearable Technology : Addressing Privacy Security Concerns without Derailing Innovation ,21 RICH. J.L TECH.6(2015).
- [3] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash. Internet of things : A survey on enabling technologies, protocols, and applications. IEEE Communications Surveys Tutorials, 17(4) :2347–2376, Fourthquarter 2015. ISSN 1553-877X.
- [4] L Atzori, A Iera, and G Morabito. The Internet of Things : A survey. Computer Networks 2010.
- [5] D.Amrhein , S.Quint Cloud computing for the enterprise : Part 1 : capturing the cloud, developer works. IBM Report,2009
- [6] M.Islam NAAS , Placement des données de l’Internet des Objets dans une infrastructure de Fog , Thèse présentée et soutenue à l’Université de Bretagne Occidentale, February 2019.
- [7] V.Gezer, Jumyung Um, and Martin Ruskowski. An introduction to edge computing and a real-time capable server architecture. Int. J. Adv. Intell. Syst.(IARIA), 11(7) :105–114, 2018.
- [8] A.Settouti, Y. Khalid , Indexation et selection des services Cloud Computing IaaS pour des reseaux de capteurs, PhD thesis, Université Abou-Bekr Belkaid – TLEMCEM, March 2018.
- [9] K. Bessai, Gestion optimale de l’allocation des ressources pour effectuer des opérations dans le cadre du cloud, Publié decembre 2014.
- [10] B. Amin, Sur l’évaluation des performances du Cloud Mesures d’environnements de calcul à l’aide de PRISM Vérificateur de modèle , University Mohamed Boudiaf - M’SILA , mémoire de Master,2016/2017.
- [11] H. Matallah. Vers un nouveau modèle de stockage et d’accès aux données dans les Big Data et les Cloud Computing. PhD thesis, Université de Tlemcen, Algérie, 2018.

- [12] D.Evans The internet of things how the next evolution of the internet is changing everything. Cisco White Paper, 2017.
- [13] F.Alghamdi, Saoucene Mahfoudh, and Ahmed Barnawi, "A Novel Fog Computing Based Architecture to Improve the Performance in Content Delivery Networks", *Wireless Communications and Mobile Computing* Volume 2019, Article ID 7864094.
- [14] NIST , The NIST Definition of Fog Computing, 2018.
- [15] L.Hardesty Fog Computing group publishes reference architecture , 2017.
- [16] NIST, The NIST definition of fog computing, NIST Special Publ 800-191 ,2017.
- [17] Z.Mahmood Cloud computing : characteristics and deployment approaches. In : 11th IEEE international conference on computer and information technology , 2011.
- [18] P.Varshney , Y.Simmhan Demystifying fog computing : characterizing architectures, applications and abstractions. In : E/ACM 1st international conference on fog and edge computing (ICFEC), IEEE 2017.
- [19] Cisco , Fog computing and the internet of things : extend the cloud to where the things are, 2015.
- [20] H. Berndt, P. Graubmann, and M. Wakano, "Service specification concepts in TINA-C" in *Towards a Pan-European Telecommunication Service Infrastructure—ISN '94*, H.-J. Kugler, A. Mullery, and N. Niebert, Eds. Heidelberg, Germany : Springer, 1994, pp. 355–366.
- [21] K. Hong, D. Lillethun, U. Ramachandran, B. Ottenwälder, and B. Koldehofe, "Mobile fog : A programming model for large-scale applications on the Internet of Things," in *Proc. 2nd ACM SIGCOMM Workshop Mobile Cloud Comput.*, Hong Kong, 2013, pp. 15–20.
- [22] N. K. Giang, M. Blackstock, R. Lea, and V. C. M. Leung, "Developing IoT applications in the fog : A distributed dataflow approach," in *Proc. 5th Int. Conf. Internet Things (IOT)*, Seoul, South Korea, 2015, pp. 155–162.
- [23] L. F. Bittencourt, M. M. Lopes, I. Petri, and O. F. Rana, "Towards virtual machine migration in fog computing," in *Proc. 10th Int. Conf. P2P Parallel Grid Cloud Internet Comput. (3PGCIC)*, Kraków, Poland, 2015, pp. 1–8.
- [24] V. Cardellini, V. Grassi, F. L. Presti, and M. Nardelli, "On QoS-aware scheduling of data stream applications over fog computing infrastructures," in *Proc. IEEE Symp. Comput. Commun. (ISCC)*, Larnaca, Cyprus, 2015, pp. 271–276.
- [25] A. Kapsalis, P. Kasnesis, I. S. Venieris, D. I. Kaklamani, and C. Z. Patrikakis, "A cooperative fog approach for effective work- load balancing," *IEEE Cloud Comput.*, vol. 4, no. 2, pp. 36–45, Mar./Apr. 2017.

- [26] H. Shi, N. Chen, and R. Deters, "Combining mobile and fog computing : Using CoAP to link mobile device clouds with fog computing," in Proc. IEEE Int. Conf. Data Sci.
- [27] M. Slabicki and K. Grochla, "Performance evaluation of CoAP, SNMP and NETCONF protocols in fog computing architecture," in Proc. IEEE/IFIP Netw. Oper. Manag. Symp. (NOMS), Istanbul, Turkey, 2016, pp. 1315–1319.
- [28] Y. N. Krishnan, C. N. Bhagwat, and A. P. Utpat, "Fog computing—Network based cloud computing," in Proc. 2nd Int. Conf. Electron. Commun. Syst. (ICECS), Coimbatore, India, 2015, pp. 250–251.
- [29] M. Aazam and E.-N. Huh, "Fog computing and smart gateway based communication for cloud of things," in Proc. Int. Conf. Future Internet Things Cloud (FiCloud), Barcelona, Spain, 2014, pp. 464–470.
- [30] R. Moreno-Vozmediano, R. S. Montero, E. Huedo, and I. M. Llorente, "Cross-site virtual network in cloud and fog computing," IEEE Cloud Comput., vol. 4, no. 2, pp. 46–53, Mar./Apr. 2017.
- [31] M. Zhanikeev, "A cloud visitation platform to facilitate cloud federation and fog computing," Computer, vol. 48, no. 5, pp. 80–83, May 2015.
- [32] K. S. Awaisi et al., "Towards a Fog Enabled Efficient Car Parking Architecture," IEEE Access, vol. 7, pp. 159100–159111, Nov. 2019.
- [33] H. Gupta, A. V. Dastjerdi, S. K. Ghosh, R. Buyya. ifogsim : A toolkit for modeling and simulation of resource management techniques in the internet of things, edge and fog computing environments. Software : Practice and Experience, 47(9) :1275–1296, 2017.
- [34] F. Luiz Bittencourt, J. Diaz-Montes, R. Buyya, O. F. Rana, et M. Parashar. Mobility-aware application scheduling in fog computing. IEEE Cloud Computing, 4(2) :26–35, 2017.
- [35] S. Agarwal, S. Yadav, and A. K. Yadav, "An efficient architecture and algorithm for resource provisioning in fog computing," Int. J. Inf. Eng. Electron. Bus., vol. 8, no. 1, pp. 48–61, 2016.
- [36] S. Yangui et al., "A platform as-a-service for hybrid cloud/fog environments," in Proc. IEEE Int. Symp. Local Metropolitan Area Netw. (LANMAN), Rome, Italy, 2016, pp. 1–7.
- [37] C. Mouradian, D. Naboulsi, S. Yangui, R. H. Glitho, M. J. Morrow and P. A. Polakos, "A Comprehensive Survey on Fog Computing : State-of-the-Art and Research Challenges," in IEEE Communications Surveys Tutorials, vol. 20, no. 1, pp. 416–464, Firstquarter 2018, doi : 10.1109/COMST.2017.2771153.

- [38] A. Brogi, S. Forti, QoS-aware deployment of IoT applications through the fog. *IEEE Internet Things J.* 2017, 4, 1185–1192.
- [39] T. Qayyum, Malik, A.W.; Khattak, M.A.K.; Khalid, O.; Khan, S.U. FogNetSim++ : A toolkit for modeling and simulation of distributed fog environment. *IEEE Access* 2018, 6, 63570–63583.
- [40] A. Varga, Hornig, R. An overview of the OMNeT++ simulation environment. In *Proceedings of the 1st International Conference on Simulation Tools and Techniques for Communications, Networks and Systems Workshops*, Marseille, France, 3–7 March 2008.
- [41] X. Masip-Bruin, E. Marín-Tordera, A. Alonso, and J. Garcia, “Fogto-cloud Computing (F2C) : The key technology enabler for dependable e-health services deployment,” in *2016 Mediterranean Ad Hoc Networking Workshop (Med-Hoc-Net)*, 2016, pp. 1–5.
- [42] A. Monteiro, H. Dubey, L. Mahler, Q. Yang, and K. Mankodiya, “Fit : A Fog Computing Device for Speech Tele-Treatments,” in *2016 IEEE International Conference on Smart Computing (SMARTCOMP)*, 2016, pp. 1–3.
- [43] H. Dubey, J. Yang, N. Constant, A. M. Amiri, Q. Yang, and K. Makodiya, “Fog Data : Enhancing Telehealth Big Data Through Fog Computing,” in *Proceedings of the ASE BigData SocialInformatics 2015*, New York, NY, USA, 2015, p. 14 :1–14 :6.
- [44] X. Hou, Y. Li, M. Chen, D. Wu, D. Jin, and S. Chen, “Vehicular Fog Computing : A Viewpoint of Vehicles as the Infrastructures,” *IEEE Trans. Veh. Technol.*, vol. 65, no. 6, pp. 3860–3873, Jun. 2016.
- [45] S. K. Datta, C. Bonnet, and J. Haerri, “Fog Computing architecture to enable consumer centric Internet of Things services,” in *2015 IEEE International Symposium on Consumer Electronics (ISCE)*, 2015, pp. 1–2.
- [46] N. B. Truong, G. M. Lee, and Y. Ghamri-Doudane, “Software defined networking-based vehicular Adhoc Network with Fog Computing,” in *2015 IFIP/IEEE International Symposium on Integrated Network Management (IM)*, 2015, pp. 1202–1207.
- [47] J. Li, J. Jin, D. Yuan, M. Palaniswami, and K. Moessner, “EHOPES : Data-centered Fog platform for smart living,” in *Telecommunication Networks and Applications Conference (ITNAC)*, 2015 International, 2015, pp. 308–313.
- [48] Y. Yan and W. Su, “A fog computing solution for advanced metering infrastructure,” in *2016 IEEE/PES Transmission and Distribution Conference and Exposition (T D)*, 2016, pp. 1–4.

- [49] B. Tang, Z. Chen, G. Hefferman, T. Wei, H. He, and Q. Yang, "A Hierarchical Distributed Fog Computing Architecture for Big Data Analysis in Smart Cities," in *Proceedings of the ASE BigData SocialInformatics 2015*, New York, NY, USA, 2015, p. 28 :1–28 :6.
- [50] V. Gazis, A. Leonardi, K. Mathioudakis, K. Sasloglou, P. Kikiras, and R. Sudhaakar, "Components of fog computing in an industrial internet of things context," in *2015 12th Annual IEEE International Conference on Sensing, Communication, and Networking - Workshops (SECON Workshops)*, 2015, pp. 1–6.
- [51] Y. Xu, V. Mahendran, and S. Radhakrishnan, "Towards SDN-based fog computing : MQTT broker virtualization for effective and reliable delivery," in *2016 8th International Conference on Communication Systems and Networks (COMSNETS)*, 2016, pp. 1–6.
- [52] M. Aazam and E.-N. Huh, "E-HAMC : Leveraging Fog computing for emergency alert service," in *2015 IEEE International Conference on Pervasive Computing and Communication Workshops (PerCom Workshops)*, 2015, pp. 518–523.
- [53] Shruti and S.Rani , "Proceedings of the Third International Conference on Information Management and Machine Intelligence" , in 2023 Springer Nature Singapore , januray 2023
- [54] IoT Editorial Office. Acknowledgment to the Reviewers of IoT in 2022. *IoT*. 2023 ; Vol. 4(1) pp. 56.

Résumé

L'industrie automobile se développe de jour en jour et les véhicules personnels sont de plus en plus important pour le transport. Avec l'augmentation des véhicules de transport privés, obtenir une place gratuite pour garer sa voiture, en particulier dans les zones peuplées, est non seulement devenue difficile, mais entraîne également plusieurs problèmes, tels que : les embouteillages, la perte de temps, la pollution de l'environnement, et surtout la consommation de carburant inutile. D'autre part, les places de stationnement automobile dans les zones urbaines n'augmentent pas au même rythme que les véhicules sur les routes. Par conséquent, les systèmes de stationnement intelligents sont devenus un élément essentiel pour résoudre les problèmes mentionnés ci-dessus.

Plusieurs chercheurs ont tenté d'automatiser la stationnement répartition de l'espace en utilisant des technologies de pointe. Des travaux importants ont été réalisés dans les domaines des réseaux de capteurs sans fil (WSN), du Cloud Computing, du Fog Computing et de l'Internet des objets (IoT) pour faciliter les progrès des services de stationnement intelligents. Peu de chercheurs ont proposé des méthodes intelligentes stationnement en utilisant les infrastructures de Cloud Computing.

Cependant, la latence est une préoccupation importante dans les applications basées sur le Cloud, y compris le transport intelligent et en particulier dans les systèmes de stationnement intelligents. Le Fog Computing amène les ressources de Cloud Computing à proximité immédiate de la périphérie du réseau, surmonte non seulement le problème de latence, mais fournit également des améliorations significatives. La principale motivation pour utiliser le calcul du brouillard dans ce travail est de minimiser la latence ainsi que l'utilisation du réseau dans le système global de stationnement intelligent. Afin de mesurer l'efficacité de notre contribution nous avons utiliser iFogSim comme outil de simulation de cet environnement.

Les mots-clés : Fog Computing, IoT, IfogSim, latence, utilisation du réseau.

Summary

The automotive industry is growing rapidly, and personal vehicles have become increasingly important for transportation. With the increasing number of private vehicles, finding a free parking space for cars, especially in densely populated areas, has become not only difficult but also leads to several problems such as traffic congestion, time loss, environmental pollution, and unnecessary fuel consumption. On the other hand, parking spaces in urban areas are not increasing at the same rate as the number of vehicles on the roads. Therefore, smart parking systems have become essential to address the aforementioned issues.

Several researchers have attempted to automate parking space allocation using advanced technologies. Significant work has been done in the fields of Wireless Sensor Networks (WSN), Cloud Computing, Fog Computing, and the Internet of Things (IoT) to facilitate advancements in smart parking services. While few researchers have proposed intelligent parking methods using Cloud Computing infrastructure, latency remains a significant concern in Cloud-based applications, including smart transportation and particularly in smart parking systems. Fog Computing brings Cloud Computing resources in close proximity to the network edge, overcoming not only the latency issue but also providing significant improvements.

The main motivation for using Fog Computing in this work is to minimize latency and network usage in the overall smart parking system. To measure the effectiveness of our contribution, we have used iFogSim as a simulation tool for this environment.

Keywords : Fog Computing, IoT, IfogSim, Latency, Network usage.

الملخص

تتطور صناعة السيارات بسرعة، وأصبحت المركبات الشخصية ذات أهمية متزايدة في وسائل النقل. مع زيادة عدد المركبات الخاصة، أصبح العثور على موقف مجاني للسيارات، خاصة في المناطق المكتظة بالسكان، ليس صعبًا فحسب، بل يتسبب أيضًا في عدة مشاكل مثل ازدحام المرور، فقدان الوقت، تلوث البيئة، واستهلاك غير ضروري للوقود. من ناحية أخرى، لا تزيد أماكن وقوف السيارات في المناطق الحضرية بنفس معدل زيادة عدد المركبات على الطرق. لذا، أصبحت أنظمة وقوف السيارات الذكية أمرًا ضروريًا لمعالجة المشكلات المذكورة. قام العديد من الباحثين بمحاولة توفير تخصيص آلي لمساحات وقوف السيارات باستخدام التكنولوجيا المتقدمة. تم إجراء أعمال مهمة في مجالات شبكات الاستشعار اللاسلكية (WSN) والحوسبة السحابية والحوسبة الضبابية وإنترنت الأشياء (IoT) لتسهيل التطورات في خدمات وقوف السيارات الذكية. في حين اقترح قلة من الباحثين طرق ذكية لوقوف السيارات باستخدام بنية الحوسبة السحابية، إلا أن التأخير لا يزال قلقًا كبيرًا في التطبيقات المعتمدة على السحابة، بما في ذلك التنقل الذكي وخاصة في أنظمة وقوف السيارات الذكية. تجلب الحوسبة الضبابية موارد الحوسبة السحابية على مقربة من حافة الشبكة، مما يتغلب ليس فقط على مشكلة التأخير، بل يوفر أيضًا تحسينات كبيرة. الدافع الرئيسي لاستخدام الحوسبة الضبابية في هذا العمل هو تقليل التأخير واستخدام الشبكة في النظام الشامل لوقوف السيارات الذكية. لقياس فعالية مساهمتنا، استخدمنا iFogSim كأداة محاكاة لهذا البيئة. **الكلمات الرئيسية:** الحوسبة الضبابية، إنترنت الأشياء (IoT)، iFogSim، التأخير، استخدام الشبكة